

AI-Powered Legal Document Analysis System:
Clause Detection, Category-Level Risk Prioritization
and Plain-Language Explanation on CUAD

by

Jerin Aktar

22101279

Afnan Mazumdar

24141229

Shoyeb Hasan Sayem

22101386

A project submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
July 2026

© 2026. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The project submitted is our own original work while completing degree at Brac University.
2. The project does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The project does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

Jerin Aktar

22101279

Afnan Mazumdar

24141229

Shoyeb Hasan Sayem

22101386

Approval

The project titled “AI-Powered Legal Document Analysis System: Clause Detection, Category-Level Risk Prioritization and Plain-Language Explanation on CUAD” submitted by

1. Jerin Aktar (22101279)
2. Afnan Mazumdar (24141229)
3. Shoyeb Hasan Sayem (22101386)

of Summer, 2026 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science and Engineering in July 2026.

Examining Committee:

Supervisor:
(Member)

Utsho Kumar Roy

Designation
Department of Computer Science and Engineering
Brac University

Head of Department:
(Chair)

Sadia Hamid Kazi, PhD

Chairperson and Associate Professor
Department of Computer Science and Engineering
Brac University

Ethics Statement

This project uses the Contract Understanding Atticus Dataset (CUAD), a public research dataset of 510 business contracts taken from public filings and labelled by legal experts for The Atticus Project. We did not collect or use any private, confidential or personal documents, and no people took part as research subjects. All experiments ran on our own computers. The outside models we fine-tuned (DistilBERT, FLAN-T5) and all the software libraries are used under their open-source licences, and every outside source is cited.

The system is a research prototype meant to help with contract review. It is not legal advice, and the app says so. The ethical risks of relying on its output (false comfort from missed clauses, too much trust by non-lawyers, rules that differ between countries, whose side the risk is judged from, the fact that no lawyer has reviewed the risk levels, and privacy in real use) are discussed in Section 3.4.

Abstract

This project builds and tests a system of three models for reviewing contracts automatically, using CUAD (510 contracts, 41 clause categories) and a held-out test set made of 20% of the contracts. A DistilBERT model that reads the contract in small windows *loses* to a simple TF-IDF model that reads the whole document (micro-F1 0.692 against 0.775), and the gap remains after training on all 53,662 windows, with three random seeds, six ways of combining window scores and a range of thresholds. An ensemble in which both models must agree reaches 0.779, but its lead is within the bootstrap and split variation. Extracting the clause text reaches a token-F1 of 0.764 when the model is given the right window. The fine-tuned summarizer outputs one of 41 sentences we wrote ourselves, word for word, for every input, and scores 0.116 against clause-specific references. Over the whole system only 22.0% of real clauses get through all three steps, and the setup with the best overall score misses 36.9% of High-risk clauses.

Keywords: legal NLP; CUAD; clause extraction; category-level risk prioritization; contract summarization; held-out evaluation

Acknowledgement

First of all, all praise to the Almighty for allowing us to complete this project without any major problem. We thank our supervisor, Utsho Kumar Roy, for his guidance and patience in every part of this work, and especially for insisting on a strict held-out test: that one rule shaped how honestly this report presents its results. Finally, we thank our families for their constant support. Without them, this work would not have been possible.

Table of Contents

Declaration	i
Approval	ii
Ethics Statement	iii
Abstract	iv
Acknowledgement	v
Table of Contents	vi
List of Figures	x
List of Tables	xi
Nomenclature	xii
1 Introduction	1
1.1 Background	1
1.1.1 System at a Glance	1
1.2 Rationale of the Study or Motivation	1
1.3 Problem Statement	2
1.3.1 Research Questions	3
1.4 Objective	3
1.5 Methodology in Brief	4
1.6 Scopes and Challenges	4
1.7 Report Organization	5
2 Literature Review	6
2.1 Preliminaries	6

2.1.1	Transformer Encoders and Model Distillation	6
2.1.2	Long-Document Strategies and Multiple-Instance Learning . .	6
2.1.3	Text-to-Text Models for Summarization	7
2.2	Review of Existing Research	8
2.2.1	Legal NLP and the CUAD Benchmark	8
2.2.2	Legal Risk Assessment and Decision Support	8
2.2.3	Plain-Language Legal Communication	9
2.2.4	Faithfulness and Hallucination in Generated Summaries . . .	10
2.3	Summary of Key Findings	10
3	Requirements, Impacts and Constraints	12
3.1	Final Specifications and Requirements	12
3.1.1	Functional requirements	12
3.1.2	Non-functional requirements	12
3.1.3	Data and resource requirements	13
3.1.4	Design constraints	13
3.2	Societal Impact	14
3.3	Environmental Impact	14
3.4	Ethical Issues	15
3.4.1	Data Handling and Privacy	16
3.5	Standards	16
3.6	Project Management Plan	17
3.6.1	Schedule	17
3.6.2	Budget	17
3.6.3	Resource management	17
3.7	Risk Management	18
3.8	Economic Analysis	18
4	Proposed Methodology	21
4.1	Design Process or Methodology Overview	21
4.2	Preliminary Design or Design (Model) Specification	22
4.2.1	Stage 2A: Presence Classification	22
4.2.2	Stage 2B: Span Extraction	25
4.2.3	Stage 1: Plain-Language Summarization	26
4.2.4	Category-Level Risk Prioritization	26

4.2.5	Leakage Discipline	29
4.3	Data Collection	29
4.3.1	Data Cleaning	30
4.3.2	Data Transformation	30
4.3.3	Data Integration	31
4.3.4	Data Reduction	31
4.3.5	Summary of Preprocessed Data	31
4.4	Implementation of Selected Design	33
4.4.1	Validation Protocol, and Where the Hyperparameters Came From	34
4.4.2	Optimization Schedule	35
4.4.3	Training the Presence Classifier	36
4.4.4	Training the Span Extractor	37
4.4.5	Training the Summarizer	37
4.4.6	Engineering Notes: Training on Consumer Hardware	37
4.4.7	Measured: What Sparse Attention Would Have Cost	38
4.4.8	End-to-End Data Flow	40
4.4.9	The Clausify Application	40
5	Result Analysis	47
5.1	Performance Evaluation	47
5.1.1	Presence Classification	47
5.1.2	Span Extraction	50
5.1.3	Summarization	52
5.2	Analysis of Design Solutions	54
5.2.1	End-to-End: What Actually Reaches the User	54
5.2.2	Risk-Weighted Evaluation: Does the Metric Match the Pur- pose?	56
5.2.3	What the errors actually look like	58
5.2.4	Does the AND rule punish rare categories harder than com- mon ones?	58
5.3	Final Design Adjustments	61
5.3.1	Ablation: was max-pooling the right aggregation rule?	61
5.3.2	The Threshold and What the Confidence Score Means	62
5.4	Statistical Analysis	64

5.4.1	How much of that ranking is real?	64
5.4.2	How much of this moves if we only change the random seed? .	65
5.4.3	Was the transformer simply undertrained?	66
5.4.4	A third source of variance: the split itself	68
5.5	Comparisons and Relationships	68
5.6	Discussions	69
5.6.1	Usability	70
5.6.2	Honest Limitations of the Demo	70
6	Conclusion	72
6.1	Summary of Findings	72
6.2	Contributions to the Field	73
6.3	Threats to Validity	74
6.3.1	Internal validity: could our own procedure have produced these numbers?	74
6.3.2	Construct validity: do our metrics measure what we claim? .	75
6.3.3	External validity, where do these results transfer?	76
6.3.4	Statistical conclusion validity: are the intervals trustworthy? .	76
6.4	Limitations	76
6.5	Recommendations for Future Work	77
	Bibliography	80
A	Full Risk Taxonomy	82
B	Reproducibility Checklist	84
C	Repository and Artifact Layout	88

List of Figures

1.1	Three questions the system answers	2
4.1	System pipeline	21
4.2	Retrieval ceiling	23
4.3	Sliding-window generation	24
4.4	Multiple-instance max-pooling	24
4.5	Ensemble rules	25
4.6	Presence vs. focus window	26
4.7	Risk-level distribution	29
4.8	Per-category split balance	33
4.9	Learning-rate schedules	36
4.10	Presence classifier loss	37
4.11	Summarizer loss	38
4.12	Pipeline data flow	40
4.13	Clausify input screen	41
4.14	Highlighted contract and checklist	42
4.15	Clausify results view	43
4.16	High-risk clause cards	44
5.1	Presence model micro-F1	48
5.2	Confusion matrix heatmap	49
5.3	Top-12 per-category F1	51
5.4	Threshold and calibration	62

List of Tables

2.1	Related work	11
3.1	Resource requirements	13
3.2	Project schedule	18
3.3	Risk management plan	19
3.4	Cost–benefit comparison	19
4.1	Retrieval hit-rates	23
4.2	Train/test split	32
4.3	Training configurations	33
4.4	Validation arrangements	35
4.5	Longformer vs. DistilBERT cost	39
4.6	Application latency	45
5.1	Presence model comparison	48
5.2	AND-ensemble confusion matrix	49
5.3	Confusion-matrix metrics	49
5.4	Categories with zero F1	50
5.5	Span extraction results	51
5.6	Summarizer ROUGE-L	53
5.7	Fine-tuned vs. zero-shot output	53
5.8	End-to-end clause survival	55
5.9	Span-stage loss breakdown	55
5.10	Results by risk band	57
5.11	Representative errors	59
5.12	Rare vs. common categories	60
5.13	Aggregation ablation	61
5.14	Cluster bootstrap results	64

5.15	Seed variance	65
5.16	Training-budget control	67
5.17	Split variance	68
5.18	Final test results	69
A.1	Full risk taxonomy	82

Nomenclature

The list below gives the abbreviations used in this report.

BERT	Bidirectional Encoder Representations from Transformers
CI	Confidence Interval
CPU / GPU	Central Processing Unit / Graphics Processing Unit
CUAD	Contract Understanding Atticus Dataset
ECE	Expected Calibration Error
EM	Exact Match
FN / FP	False Negative / False Positive
IP	Intellectual Property
LLM	Large Language Model
LR	Logistic Regression
MIL	Multiple-Instance Learning
MPS	Metal Performance Shaders (Apple GPU backend)
NLP	Natural Language Processing
QA	Question Answering
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
RQ	Research Question
SD	Standard Deviation
SEC	US Securities and Exchange Commission
SQuAD	Stanford Question Answering Dataset
TF-IDF	Term Frequency-Inverse Document Frequency
TN / TP	True Negative / True Positive

Glossary of terms

AND-ensemble A rule that reports a clause category only when both the TF-IDF model and the DistilBERT model say it is present.

Bootstrap A way to measure uncertainty: the test contracts are drawn again at random, with repeats, many times, and the score is recomputed each time.

Calibration Whether a model’s score can be read as a real chance. A well-calibrated score of 0.8 is right about 80% of the time.

Clause category One of the 41 types of contract clause labelled in CUAD (for example *Governing Law*, *Non-Compete*, *Cap on Liability*).

Contract-level split A train/test split that puts each whole contract on one side, so no clause from a training contract can appear in the test set.

Max-pooling (over windows) Scoring every window of a document and using the highest window score as the score for the whole document.

Micro-F1 One F1 score computed over all (contract, category) decisions together, so every decision counts the same.

Presence classification Deciding, for each clause category, whether a contract contains at least one clause of that category.

Retrieval ceiling The most a “search first” design can ever find: a clause in a window that the search never picks can never be found.

Span extraction Finding the exact text of a clause inside the contract, treated as a question-answering task.

Token-F1 How much the extracted text overlaps the real clause, word by word (1.0 means a perfect match).

Weaker party The person or company signing the contract with less bargaining power; the risk table is written from their side.

Chapter 1

Introduction

1.1 Background

Contracts are part of almost everything people and businesses do, but reading them takes a lot of time. A typical contract in our dataset is about 33,000 characters long (roughly 5,000 words), and the longest is more than 338,000 characters. Somewhere in all that text are the parts that really matter: who owns new intellectual property, whether there is a limit on how much you can owe, whether the contract renews on its own, and whether you are stopped from working with competitors after it ends. Having a lawyer read a contract takes hours and costs money that many people and small businesses cannot afford. So people often sign contracts they do not fully understand and take on risks they never noticed.

The Contract Understanding Atticus Dataset (CUAD) [1] was created to help automate this kind of review. It has 510 real business contracts in which legal experts marked every clause that belongs to one of 41 important categories. This marking work is said to be worth about two million US dollars of lawyers' time. However, CUAD only treats the problem as finding text: given a category, find the clause. Once it finds a clause such as "Uncapped Liability", it stops. It does not warn the reader what the clause means for them, it does not explain it in simple words, and it does not check how clauses affect each other.

1.1.1 System at a Glance

Chapter 4 explains the system in detail. Figure 1.1 shows the whole system in one picture. The system answers three questions about a contract, in order: *what is in it*, *where is it*, and *what does it mean*. CUAD was built to test only the first two. The third step, shaded in the figure, is what this project adds.

1.2 Rationale of the Study or Motivation

Our project tries to close the gap between finding a clause and understanding it. On top of finding clauses, we add two things. First, a simple risk table that gives



Figure 1.1: Three questions the system answers

every clause *category* a High, Medium or Low level, judged from the side of the weaker party. Second, a text model that is meant to turn each clause into one plain-English sentence. We built the clause-finding models and the risk table into *Clausify*, a prototype web app: a user pastes or uploads a contract and gets back a list of clauses sorted by risk. Both added parts worked less well than we hoped. The risk table has not been checked by a lawyer (Section 4.2.4), and the text model learned to repeat a fixed sentence for each category instead of describing the actual clause (Section 5.1.3). Chapters 5 and 6 report these results honestly.

So the reason for this study is practical as well as academic. Earlier work on CUAD answers the question “where is the clause?” for a lawyer who already knows what to look for. The people who need help most (individuals, freelancers and small businesses who sign contracts without a lawyer) also need to know which clauses matter most to them and what each one means in everyday language. A system that gives those answers, and is honest about how often it is wrong, could make contract review possible for people who cannot pay for it today. Just as important, we wanted to measure how well such a system really works instead of assuming it. A contract-review tool that looks confident but quietly misses clauses is worse than having no tool at all.

1.3 Problem Statement

In short, the project works on four connected problems:

1. **Finding which clauses are present.** For each of the 41 CUAD categories, decide whether a contract has at least one clause of that type. The main difficulty is length: a typical contract is about $16\times$ longer than the amount of text a standard transformer model can read at once.
2. **Finding the exact clause text.** For each category that is present, find the exact words of the clause in the contract. We treat this as a question-answering task, in the same style as SQuAD [2].
3. **Explaining in plain language.** Rewrite each clause as one sentence that a non-lawyer can understand.
4. **Showing the risk.** Give every clause found a risk level and a one-line reason, so the result reads like a ranked list of warnings instead of a list of quotes. We did not test whether this actually helps users (Section 6.3).

1.3.1 Research Questions

Our objectives turn into five research questions, which the rest of this report answers. We give the answers here so readers can check each claim against the evidence instead of searching for it.

RQ1 *Can a transformer that reads a contract in small windows beat a simple word-based model that reads the whole contract, at finding which clauses are present?*

No. The simple model wins by +0.082 micro-F1. The gap stays even when the transformer is trained on all its training data (Section 5.4.3) and with three different random seeds (Section 5.4.2). Answered in Chapter 5.

RQ2 *How well can the system find the exact clause text in long contracts?*

Token-F1 is 0.764 (95% CI [0.743, 0.782]) *when the model is given a window that contains the clause*, but only 0.383 when the window is chosen by the first model (Sections 5.1.2 and 5.2.1).

RQ3 *Does the summarizer really describe each specific clause in plain English?*

No. For every input it copies one of 41 sentences we wrote in advance. Against clause-specific reference sentences, it scores no better than just printing that fixed sentence without any model (Section 5.1.3).

RQ4 *How do errors add up across the whole system?*

They multiply, and it is worse than the separate scores suggest: only 22.0% of real clauses get through all three steps, compared with the $\approx 65\%$ that the separate scores would predict (Section 5.2.1).

RQ5 *Do the system’s mistakes avoid the clauses that would hurt a signer most?*

No, the opposite. The setup we report as our main result misses 36.9% of High-risk clauses, while the model we call weaker misses only 15.9% (Section 5.2.2). The overall score and the harm to the user point in different directions.

Three of these five answers are negative. We report them as real results of the project, not as failures. Each one is something a reader would have got wrong by looking only at the separate model scores.

1.4 Objective

- Build and fairly test a complete three-model CUAD system on an ordinary laptop, using an 80/20 split by contract so that every reported number comes from contracts the models have never seen.
- Check whether a transformer really beats a strong simple model at finding which clauses are present, instead of assuming it does.
- Measure how well the exact clause text is found, using token-level F1 and exact match on the test set.

- Test the summarizer against two different sets of reference sentences, because one automatic score turned out to measure the reference sentences rather than the task.
- Measure how many clauses survive the whole system from start to finish, not only the score of each step.
- Put the trained models into a working demo app that shows clauses sorted by risk.

1.5 Methodology in Brief

We tested every design before building on it, using one fixed 80/20 split of CUAD by contract (408 training contracts and 102 test contracts). After exploring the data (Section 4.3.5), we first built a simple word-based model (TF-IDF with logistic regression) that reads the whole contract, *before* training any transformer. We also measured the highest recall a “search first” design could ever reach (Section 4.2.1). Based on that, we trained a DistilBERT model that scores the contract in small windows and takes the highest window score as the answer for the whole contract (multiple-instance learning). We combined it with the simple model using an AND rule, which reports a clause only when both models agree. A DistilBERT question-answering model finds the exact clause text, a fine-tuned FLAN-T5-small model writes a one-sentence plain-English explanation, and a hand-made table gives each of the 41 categories a High, Medium or Low risk level.

Every model was tested once on the 102 test contracts. We report each score together with confidence intervals from a bootstrap over contracts, and we checked how the results change with the amount of training data, the random seed, the way window scores are combined, the decision threshold and the train/test split. We also tested the full system from start to finish. Finally, we put the trained models into *Clausify*, a Streamlit web app that gives a risk-sorted clause report for any contract. Chapter 4 describes the method in full.

1.6 Scopes and Challenges

Scope. The project covers English business contracts like those in CUAD: 510 contracts from US SEC filings, marked for 41 clause categories. Within that scope it handles four tasks (finding which clauses are present, finding their exact text, explaining them in plain language and showing their risk level) and delivers a working prototype app. The following are outside the scope on purpose: legal advice of any kind; contracts in other languages or countries (including Bangladeshi contracts, which inspired the work but were not tested); judging the risk of a clause’s exact wording; finding conflicts between clauses; and a formal user study of the app. All models were sized so they can be trained and run on one ordinary laptop.

Challenges. Six challenges shaped the design:

1. **Very long documents.** A typical contract is about 33,000 characters. That

is roughly $16\times$ the 512-token limit of a standard transformer (about 2,000 characters), and the longest contract is more than 338,000 characters, about $170\times$.

2. **Very uneven categories.** Some categories appear in 100% of contracts and some in only 2.5%; a few have only a handful of examples in the whole dataset.
3. **A small, fixed dataset.** The 510 contracts are all there is, and expert marking is too costly to add more, so we had to be careful about over-fitting and about test data leaking into training.
4. **Limited hardware.** Training ran on an Apple-Silicon laptop without a CUDA GPU. Our first-choice model (DeBERTa-v3) was unstable on the MPS backend, and GPU memory was not freed between training runs.
5. **No plain-language answers to learn from.** CUAD has no plain-English versions of clauses, so we had to write our own reference sentences, and this limited what the summarizer could learn.
6. **No agreed answer for risk either.** Contract risk depends on which side you are on, the situation and the country, and there is no expert-checked risk model for CUAD’s 41 categories.

1.7 Report Organization

The rest of this report is organized as follows. Chapter 2 reviews background and earlier work. Chapter 3 sets out the system’s requirements, its effect on society, the environment, ethics and cost, and the project plan. Chapter 4 explains the method: how we designed the system, which other designs we considered, how the data was prepared, and how the models and the Clausify app were built. Chapter 5 analyses the results on the test set, including the statistics and the changes we made because of them. Chapter 6 sums up the findings, contributions and limits, and suggests future work. The appendices contain the full 41-category risk table, a checklist for repeating the work and the layout of the project files.

Chapter 2

Literature Review

2.1 Preliminaries

This section explains the models and methods the project is built on.

2.1.1 Transformer Encoders and Model Distillation

The transformer [3] is a type of neural network that reads text using self-attention instead of reading it word by word. BERT [4] showed a simple recipe that this project relies on: first train a model on a very large amount of general text, then fine-tune it for a specific task using a much smaller amount of labelled data. This matters a lot for us, because 510 contracts is a small dataset for deep learning.

Because all our work runs on one laptop, we use DistilBERT [5]. It is a smaller, six-layer version of BERT that keeps about 97% of BERT’s language ability while being 40% smaller and 60% faster. The published CUAD results use much larger models, so we give up some accuracy in exchange for being able to run everything on normal hardware. We point out this size difference whenever we discuss our results.

2.1.2 Long-Document Strategies and Multiple-Instance Learning

Contracts are much longer than a transformer can read at once. There are three usual ways to deal with this: (i) *retrieval*, where a quick, cheap method picks the few parts of the text most likely to contain the answer and only those are given to the transformer; (ii) *sliding windows*, where the model reads every part of the text and the scores are combined into one answer for the whole document; and (iii) *sparse-attention models* such as Longformer, which can read a longer piece of text at once.

The third option is often dismissed in one line as “too expensive”, but that alone is not a good reason. We did not run a Longformer experiment, so what follows is a design choice, and we call it that. The one part of it we could check cheaply on our own laptop, the cost, we did check, and Section 4.4.7 reports the result.

We had three reasons, and only the third is about cost. **First, and most important, it would not solve the problem.** Longformer can read 4,096 tokens at once. At about four characters per token, our typical contract of 33,143 characters is about 8,300 tokens and the longest is about 85,000. So Longformer would still see only half of a typical contract and one-twentieth of the longest one. It makes the window eight times wider, but that is still not wide enough for this data. We would still need windows and a way to combine their scores, which is the design we ended up using anyway, just with a much more expensive model underneath. This reason is simple arithmetic about the dataset rather than an experiment, but the arithmetic is clear: CUAD contracts are about ten times too long for a wider window to fix.

Second, availability. Our whole system uses small (distilled) models so that it can be trained and run on one laptop. We do not know of a small version of Longformer that is as reliable and well supported as DistilBERT, so using Longformer would have meant giving up the small models that make the rest of the project easy to repeat.

Third, cost in practice. We believed that fine-tuning a Longformer with 148.7M parameters at 4,096 tokens was not possible on one Apple-Silicon laptop. We were also worried about special attention code on a non-CUDA machine, after DeBERTa-v3 produced NaN (broken) values on the MPS backend (Section 4.4.6). When we later measured it (Section 4.4.7), we found the memory part of this belief was *wrong*: one training step does fit at batch size 1. The time part, however, was right by a very large margin. We kept our original reasoning in the text and reported the correction next to the measurement, instead of quietly changing the reasoning to match the result. There is also a cost when the app runs: *Clausify* scores every window of an uploaded contract for all 41 categories while the user waits, so it is only usable if this takes seconds. This can be measured without any training, so we measured it: Section 4.4.7 gives the speed and the peak memory of Longformer-base at 4,096 tokens on our laptop, next to the DistilBERT setup we actually used.

On a CUDA computer with more memory, the third reason would be weaker and the second might disappear, but the first would still hold. So this is a design choice backed by one measurement and some arithmetic. It does not prove that sparse attention cannot work on CUAD. A properly trained Longformer with windows would be a fair comparison to ask for, and we list it as future work.

Reading a document in windows and taking the highest window score is known as *multiple-instance learning* (MIL). The document is treated as a bag of windows, and the bag counts as positive if any one window is positive. Taking the maximum score does exactly this. MIL has a well-known weakness, which our experiments confirmed: it produces too many false alarms. A document has dozens of windows, so the chance that *some* window fires by mistake grows with the length of the document, and precision drops.

2.1.3 Text-to-Text Models for Summarization

T5 [6] treats every language task as “text in, text out”, which suits clause simplification well: legal text goes in and a simple sentence comes out. FLAN-T5 [7] is a

version of T5 that was trained on many tasks with instructions, so it already gives useful answers to a simple prompt such as “summarize in plain English:” without extra training. We score summaries with ROUGE-L [8], which measures how many words the output shares, in the same order, with a reference sentence. Chapter 5 shows why this score must be checked by actually reading the outputs when there are only a few reference sentences.

2.2 Review of Existing Research

This section reviews earlier work on legal language processing, legal risk, plain-language legal writing, and the accuracy of machine-written summaries.

2.2.1 Legal NLP and the CUAD Benchmark

People have long wanted to use language technology on legal text, but progress was slow for a simple reason: labelling legal text needs lawyers, and lawyers are expensive. CUAD [1], released by The Atticus Project [9] at the NeurIPS 2021 Datasets and Benchmarks track, changed this. It provides 510 real business contracts from public SEC filings, in which experienced lawyers marked every piece of text that belongs to one of 41 important clause categories, such as governing law, non-compete, limits on liability and IP assignment. The data uses the SQuAD format [2]: each (contract, category) pair is asked as a question about the contract text, and the marked clauses are the answers, with their exact character positions.

The original CUAD paper treats the task as question answering and finds that bigger models do much better. Its best results come from the extra-large DeBERTa models, and even those scores are modest, which shows how hard it is to find legal clauses. Two features of CUAD affect any model built on it. The first is that categories are very uneven: some appear in almost every contract, while others appear only about a dozen times in the whole dataset. The second is length. Contracts are far longer than a transformer can read at once, so every practical system first has to answer one question: how can a model that reads 512 tokens at a time read a 50,000-character contract?

2.2.2 Legal Risk Assessment and Decision Support

The risk part of this project has the least earlier work behind it, and that is a weakness of our work rather than of the field. Since we tell a non-lawyer which clauses to worry about, the obvious question is what this advice is based on. The honest answer is that we did not use an existing framework. We wrote the risk table ourselves (Section 4.2.4), and that is why we call it a prototype.

Earlier work does explain why this is hard. Legal language benchmarks usually treat “risk” indirectly, by sorting text into legally meaningful categories rather than judging how serious something is. LexGLUE [10] collects tasks such as predicting court decisions and classifying laws, and LEGAL-BERT [11] shows that models pre-trained on legal text do better on them. But none of these tasks asks how *dangerous*

a clause is for a particular person signing it. CUAD itself [1] says its categories are the clauses lawyers look for when checking a deal, which shows they are important without saying how important.

So how serious a clause is remains an open question, and it is hard to learn from data alone. First, risk depends on which side you are on: the same unlimited indemnity is dangerous for the party giving it and useful for the party receiving it. A risk label only makes sense once you decide whose side you are on, a problem Section 4.2.4 handles only by making an assumption. Second, risk depends on the situation: a \$50,000 limit on liability means little to a large company but can ruin a one-person business. Third, risk depends on the country: a non-compete that is binding in one country may be invalid in another. Our table gives one fixed level per category, so on purpose it ignores all three.

This has two effects on how our work should be read. First, our risk table should be compared with expert opinion, not with a published risk model, since we know of no published model that covers CUAD’s 41 categories. That expert check, as Section 4.2.4 explains, has not been done yet. Second, work on explainable legal AI argues that tools in this area should show their reasoning instead of just giving a score. Our table does this well: every level comes with a written reason that can be questioned line by line (Appendix A). This openness is the real strength of the design, but being open is not the same as being correct.

2.2.3 Plain-Language Legal Communication

Chapter 5 finds that our summarizer does not really summarize, and the relevant earlier work is about *simplifying* text, not about T5. The difference matters. Summarizing makes text shorter, while simplifying says the same thing in easier words. For a contract clause, making it shorter is often the wrong goal, because the conditions and exceptions that a summary would drop are often exactly what decides what the clause does.

The closest earlier work is Manor and Li [12], who collected plain-English summaries of contract sections and found that models trained on them tend to give general, repeated answers. This is an early version of the problem our template test measures. Measuring how easy a text is to read is a much older field [13], and it offers something our evaluation does not have: a way to check that the output really is easier to read, not just shorter. We do not report any readability score, which, looking back, is a gap, because the whole point of this part is to make clauses easier to understand.

The main lesson from this work, which our results also show, is that scores based on reference sentences measure the reference sentences. With only 41 different target sentences, ROUGE-L rewards the model for recognising the category. Section 5.1.3 measures this directly by changing the reference sentences while keeping everything else the same.

2.2.4 Faithfulness and Hallucination in Generated Summaries

When a system shows machine-written sentences next to a user’s own contract, it is a safety issue whether those sentences are true, not just a matter of quality. Research on summarization has shown that fluent machine-written summaries often say things that are not in the source. Maynez et al. [14] find that many generated summaries contain content the input does not support, and that ROUGE agrees poorly with human judgements of accuracy. Kryściński et al. [15] suggest checking factual accuracy as a separate task. Scores such as BERTScore [16] measure meaning better than simple word overlap, but they do not check facts either.

This affects our system in an unusual way. A model that only outputs fixed template sentences **cannot make things up in the usual sense**: it can only produce one of 41 sentences written by people, and each is a correct description of its category. Instead, it fails in the opposite way: it says nothing about the specific clause, and when it picks the wrong template (30% of the time, Section 5.1.3) the output is fluent and confident but about a completely different kind of clause. For a legal assistant this is arguably *worse* than a made-up detail, because it cannot be spotted from the output alone. The sentence is well written and correct in itself; only comparing it with the clause shows that it does not describe it.

It also means our evaluation is missing the checks this research recommends. We report ROUGE-L against two sets of reference sentences and we read outputs by hand. We do not report BERTScore, an automatic fact check, or a human judgement of whether the legal meaning is kept. For a part that non-lawyers will see, the last one matters most. Section 6.5 lists it as future work.

2.3 Summary of Key Findings

Earlier CUAD work stops once the clause is found. A person signing a contract probably needs more: which clauses matter, how much, and what they mean in plain language. This project tries to add that layer (a prototype risk table for each category, plain-English template sentences and a demo app). It also re-tests, on a strict test set, the common belief that a transformer must beat simple word-based methods at deciding which clauses a contract contains. We should say from the start that, of the three added parts, the results support only the demo app as something that works. The risk table has not been checked by an expert, and the plain-English part outputs category templates rather than summaries, so neither is a solved problem. What this added layer mainly gives is a measurement of how far it is from working.

Table 2.1 compares this work with the earlier work above. Two differences stand out. The obvious one is scope: earlier work stops at finding the clause text, while our output is a risk-sorted, plain-English report, which is the part a non-lawyer can act on. The less obvious one is in the “Long inputs” column. Every system on CUAD must choose how to handle long documents, but usually that choice is just stated. Here we measured it before any training: the 64.0% retrieval limit in

Section 4.2.1 is what ruled out retrieval and led us to multiple-instance learning. The Model column also shows a cost: our small models (67M parameters) are about ten times smaller than the DeBERTa-xlarge model behind CUAD’s best published result, so our extraction scores cannot be directly compared with theirs, and we do not claim they can.

Table 2.1: Related work

Work (data)	Task	Model	Long inputs	Risk	Plain English	Evaluation
Hendrycks et al. [1] CUAD, 510 contracts	Span extraction	DeBERTa-xlarge	Full-document QA with a large model	No	No	Held-out; precision at 80% recall
Chalkidis et al. [11] several legal corpora	Domain pre-training and classification	LEGAL-BERT	Truncation to 512 tokens	No	No	Per-task held-out
Chalkidis et al. [10] LexGLUE, 7 tasks	Legal language understanding	BERT and RoBERTa models	Depends on the task	Indirect (task labels)	No	Multi-task benchmark
Manor and Li [12] TL;DR-Legal style	Plain-English summarization	Seq2seq	Section-level inputs	No	Yes	ROUGE and manual review
Beltagy et al. [17] long-document benchmarks	New architecture	Longformer	Sparse attention, 4,096 tokens	No	No	Task benchmarks
Retrieval-QA pipelines various	Span extraction	Various	Retrieval then reader (ceiling usually assumed)	No	No	Various
This project CUAD, 510 contracts	Presence, span, category risk and template text	DistilBERT and FLAN-T5 (small)	Sliding-window MIL; ceiling <i>measured</i> , sparse attention <i>benchmarked</i>	Yes (category level, prototype)	Yes (template based)	Contract-level held-out; cluster bootstrap; seed, budget and split controls; end-to-end

Note. “Long inputs” is how each work handles documents longer than the encoder window; “Evaluation” is the level at which results are reported.

Chapter 3

Requirements, Impacts and Constraints

3.1 Final Specifications and Requirements

3.1.1 Functional requirements

- FR1** The system shall accept a contract as pasted text or as an uploaded `.pdf`, `.docx` or `.txt` file.
- FR2** For each of the 41 CUAD clause categories, the system shall decide whether the contract has a clause of that type and show a score.
- FR3** For each category found, the system shall show the clause text from the user's own contract.
- FR4** The system shall give every clause found a High, Medium or Low risk level and a one-line reason in plain English.
- FR5** The system shall show the clauses grouped by risk level (High first) and sorted by score within each group.
- FR6** The system shall let the user analyze either a whole contract or a list of separate clauses, and shall show the score of all 41 categories.

3.1.2 Non-functional requirements

- **Privacy.** All models shall run on the user's own computer; no contract text shall be sent to another service or saved to disk.
- **Hardware.** All models shall train and run on one ordinary laptop without a CUDA GPU.
- **Speed.** Analyzing a typical contract shall take no more than tens of seconds on a CPU (measured: median 27.1 s, worst case 32.4 s; Table 4.6).

- **Fair testing.** Every reported score shall come from contracts the models never saw during training (split by contract), and no setting shall be chosen by looking at the test set.
- **Repeatability.** The split, the baseline model and all trained models shall be saved, and a separate notebook shall recreate every reported number from them.
- **Openness.** The app shall say that it is not legal advice, and every risk level shall come with a written reason (Appendix A).

3.1.3 Data and resource requirements

Table 3.1 lists the data, hardware and software the project uses. All of it is either free to use or was already owned by the team.

Table 3.1: Resource requirements

Resource	Specification
Dataset	CUAD v1 (The Atticus Project), 510 annotated contracts: <code>master_clauses.csv</code> and <code>CUAD_v1.json</code> , obtained from the Hugging Face hub
Hardware	Apple M4 laptop, 16 GB unified memory, MPS (Metal) GPU backend; no CUDA GPU
Language and runtime	Python 3.13.5
Deep learning	PyTorch 2.8.0, Hugging Face <code>transformers</code> 5.12.1, <code>datasets</code> 5.0.0
Classical ML and data	scikit-learn 1.6.1, NumPy 2.1.3, pandas 2.2.3, <code>rouge_score</code>
Pre-trained models	<code>distilbert-base-uncased</code> (presence and span), <code>google/flan-t5-small</code> (summarizer)
Application	Streamlit (≥ 1.40) single-page web application
Development tools	Jupyter notebooks, Git and GitHub, $\LaTeX$

3.1.4 Design constraints

The requirements above meet four limits that shaped the final design. **Computing power:** one laptop cannot train very large models such as the DeBERTa-xlarge model used in the original CUAD work, so we chose smaller (distilled) models. **Data:** the 510 contracts cannot be extended, so we had to start from pre-trained models and split the data by contract. **Speed:** the app must score every window of a contract for all 41 categories, which is why it reads at most 30 windows. **Privacy:** the models must run locally, so online language-model services could not be used at all.

3.2 Societal Impact

Access to legal help. Having a lawyer review a contract takes hours and costs money that many people and small businesses cannot afford, so many people sign contracts they do not fully understand. A tool that points a non-lawyer to the clauses most likely to hurt them (unlimited liability, IP assignment, non-compete, automatic renewal) can make this easier and help people ask the right questions before signing. The main users we have in mind are freelancers, start-ups and small businesses, the groups least able to pay for professional review.

Health and safety. The system has no direct effect on physical health or safety. Its indirect effect is on money and stress: signing a contract without understanding it can lead to disputes, losses and worry, which better-informed signing could reduce.

Legal effects. The system is not legal advice, and it says so. Its biggest risk to society is *false reassurance*: a missed clause gives no warning, and no warning can be read as “nothing to worry about”. Our measurements show this risk is real. Only 22.0% of real clauses get through the whole system (Section 5.2.1), so the tool must be presented as a help for reading a contract, never as a replacement for a lawyer.

Culture and region. CUAD contains US business contracts written in English. Risk levels that make sense under US law may not apply in other countries, and contracts are written differently in different places. The work was inspired by the situation in Bangladesh, but no Bangladeshi contract was tested, so we cannot yet claim these benefits there.

3.3 Environmental Impact

The project was designed to need little computing power. All models are small (DistilBERT with 67M parameters and FLAN-T5-small). Training the three models used in the app took about 1.5 hours on a laptop, and the app runs on a CPU using at most about 1.5 GB of memory (Table 4.6).

Including the extra test runs (the full-data presence run of 140.7 minutes, two extra seeds, the evaluation runs and the Longformer test), the whole project used about ten hours of laptop computing. If a laptop uses 20–30 W under load (an estimate, not a measurement), this is well under one kilowatt-hour of electricity. That is very small compared with the many-GPU training often used for large legal language models.

Two design choices also keep the running cost low. The models run locally, so no data centre is used for each contract, and the system avoids large text-generating models, which use far more energy per request. The price is accuracy: Section 4.4.7 shows that larger models were rejected partly because of cost, and Chapter 5 reports what the small models achieve.

3.4 Ethical Issues

The Ethics Statement at the front of this report explains how the data and models were obtained. This section looks at the ethical issues that come from *using* the system, and the duties these place on the people who build it.

The system is a research prototype meant to help with contract review. It is not legal advice, and the app says so in a notice shown above every analysis. For a tool like this, the more serious ethical questions are not about the data but about what happens when someone relies on its output, so we list them clearly here.

False reassurance is the main danger. A clause the system does not find gives no warning, and no warning is easily read as “there is nothing to worry about”. Our measurements show this reading would be wrong: 21.6% of real clauses are missed at the first step, only 22.0% get through the whole system (Section 5.2.1), and the setup we report as our main result misses 36.9% of the High-risk clauses (Section 5.2.2). A user who checks a contract with this tool and sees three warnings cannot assume there are only three problems. Because missed clauses are silent while false alarms are visible, we believe the share of High-risk clauses found is the measure that matters most ethically, even though it is not the one the overall score rewards.

Too much trust by non-lawyers. The app is made for people without legal training, and they are the people least able to notice when it is wrong. Plain-English text shown with a confidence bar looks more certain than the model really is, and Section 5.3.2 shows the bar is not even a true probability: scores above 0.9 mean only about a 73% chance that the clause is really there. The plain-English reason on each card is a fixed sentence for the whole category from the risk table, not a reading of the user’s clause. The summarizer, which was meant to describe each clause, also turned out to output one fixed sentence per category (Section 5.1.3). The app labels the summarizer’s sentence clearly and warns when it belongs to a different category than the card. Shown without these warnings, both the bar and the reason would invite more trust than the evidence supports.

Different laws in different countries. CUAD follows US business law. Risk levels that are right there may be wrong elsewhere: a non-compete that is binding in one country may be invalid in another, and then a High badge would be misleading. The work was inspired by the situation in Bangladesh and we tested no Bangladeshi contract, so this is a real limitation, not just a possible one.

Whose risk is measured. The risk table assumes the user is the side with less bargaining power (Section 4.2.4). For the side that wrote the contract, the same badges are wrong in the other direction: a clause marked High for the signer may be exactly what the writer wanted. The system cannot tell which side the user is on.

Who wrote the risk levels. The risk levels were written by three computer science students and have not been checked by anyone with legal training (Section 4.2.4). Showing them in a polished app makes them look more official than they are. We believe an expert check must be done before any use beyond a demonstration.

Privacy in real use. Our experiments used only public contracts, so privacy was

not an issue here. A real deployment would face it at once, because contracts are among the most sensitive documents a business has. Section 3.4.1 explains what the demo app does and does not do with an uploaded contract, and what a real deployment would need to add.

3.4.1 Data Handling and Privacy

The app accepts contract text, and contracts are among the most sensitive documents a business has. Our experiments used only public CUAD contracts, so privacy was not an issue during this research. But the demo invites users to paste their own contracts, so we state exactly what happens to them.

What the demo does. All the models run locally: the DistilBERT models, the TF-IDF model and the FLAN-T5 summarizer are loaded from disk and run in the same program as the app, so no contract text leaves the computer the app runs on. An uploaded `.pdf`, `.docx` or `.txt` file is read into memory and turned into text on that computer. Nothing is saved to disk, no database or log keeps the text, and no usage data is collected. When the browser session ends the text is gone: it exists only in Streamlit’s memory while the page is open. The report the user can download is created on request and saved only where the user chooses.

What this depends on. This is only true because the demo runs on the user’s own computer. On a shared server, the same code would put the user’s contract in the memory of a machine they do not control, and Streamlit alone would not stop the server’s owner from reading it. The privacy comes from where the app runs, not from anything the app itself does, and we should not claim otherwise.

What is missing on purpose, and would be needed for real use. There is no login, so anyone who can reach the computer can open the app. There is no file-size limit beyond Streamlit’s default and no time limit, so a very large upload can keep the app busy. PDF and Word files are read with standard open-source libraries (`pypdf` and `python-docx`); a real deployment would need to treat these files as untrusted input, because a harmful file could try to attack the program that reads it. There is no rule for keeping data, because no data is kept. Party names are not removed before processing, which would matter if any part were ever moved to an online model service.

Our rule for any future change. As soon as any part of the system calls an online service, privacy changes completely: the contract would be sent to someone else and might be kept by them. The current design avoids this completely by using small local models. We see this as worth keeping, not just a side effect of using a laptop: it is one of the few places where the limits we worked under led to a better design.

3.5 Standards

No formal engineering standard applies to a research prototype like this, but the project followed several common practices:

- **Data format.** CUAD uses the SQuAD question-answering format [2], and our clause-text model uses the standard SQuAD start/end-token setup, so the data works with common question-answering tools.
- **Licences and credit.** The dataset, the pre-trained models and all software are used under their open-source licences (CUAD under CC BY 4.0; DistilBERT, FLAN-T5, `transformers` and Streamlit under Apache 2.0; PyTorch and scikit-learn under BSD licences), and they are credited in the references.
- **Testing practice.** We followed standard machine-learning practice: a separate test set split by contract, no tuning on the test set, confidence intervals from a bootstrap over contracts [18], and standard scores (F1, exact match, ROUGE-L [8]).
- **Repeatability.** Appendix B gives a checklist covering the software versions, data, seeds, file checksums and commands, and all code and reports are kept in Git.
- **Writing.** References use the IEEE style, and this report follows the Brac University CSE400 report template.

3.6 Project Management Plan

3.6.1 Schedule

The project was done by a team of three during 2026. Table 3.2 shows the main stages and what each one produced.

3.6.2 Budget

The project cost no money. The dataset, the pre-trained models and all software are free, and all training and testing ran on a laptop the team already owned. No cloud computing, paid services or paid labelling were used.

3.6.3 Resource management

The one shared laptop was the scarcest resource. Training runs were done one at a time, because the MPS backend does not free GPU memory between runs in the same program; the summarizer was trained on the CPU for the same reason (Section 4.4.6). Everything needed for testing (the split, the baseline model and the three trained models) was saved to disk, so tests could be repeated without retraining. Code, notebooks and reports were kept in Git.

Table 3.2: Project schedule

Period (2026)	Activities	Deliverables
Before June	Problem definition, literature review, initial design including a cross-clause rule engine	Project proposal
June	Data acquisition, wide-to-long preprocessing, exploratory data analysis	EDA notebook
July	Contract-level 80/20 split; TF-IDF baseline; training of the presence, span and summarizer models; standalone evaluation; Clausify application	Trained models, <code>test.ipynb</code> , Clausify prototype
August	Supervisor review: training-budget and seed controls, cluster bootstrap, span and end-to-end evaluation; second review: risk-weighted evaluation, threshold and calibration, aggregation ablation, split variance	Project and findings reports, analysis scripts
September	Write-up of the full report and a conference-style paper; presentation	Report, paper, presentation slides
October	Viva preparation; re-training and re-testing of each model; final report in the CSE400 template	Final project report

3.7 Risk Management

Table 3.3 lists the main risks we found during the project and what we did about each one. Several of them actually happened, and the last column shows what was done.

3.8 Economic Analysis

Development cost. The project cost no money: the data, models and software are free, and the laptop was already owned (Section 3.6). The real cost was the team’s time and about ten hours of laptop computing.

Running cost. The models run on a CPU with about 1.5 GB of memory, so the system can run on ordinary hardware with no cost per contract. Table 3.4 compares this with the two options a user has today.

Benefits. The main benefit is quick sorting: pointing the reader to the parts of

Table 3.3: Risk management plan

Risk	Impact	Mitigation
First-choice backbone unstable on available hardware	High	Occurred (DeBERTa-v3 produced NaN gradients on MPS). Switched to DistilBERT; the method is unchanged and the backbone can be swapped back on CUDA hardware.
GPU memory exhaustion during training	Medium	Occurred. One training run per process; summarizer trained on CPU.
Data leakage between train and test	High	Contract-level split reused by all three models; retrieval and inference never use gold offsets; ensemble is parameter-free (Section 4.2.5).
Optimistic results from test-set tuning	High	No hyperparameter was tuned; all settings are published defaults or fixed by arguments made before training (Section 4.4.1).
Over-claiming small differences between models	Medium	Cluster-bootstrap confidence intervals, seed, budget and split controls (Section 5.4.1 onwards).
An architecture that cannot reach the required recall	High	Retrieval ceiling measured before training (64%), which ruled out the retrieval design at no training cost (Section 4.2.1).
Schedule overrun from long training runs	Medium	Distilled models (about 1.5 hours total training); subsampled training sets; saved artifacts so evaluation needs no retraining.
Loss of work or results	Medium	Version control on GitHub; trained weights backed up before any re-training.
Users over-trusting the application	High	Not-legal-advice notice; risk badge presented as more reliable than quoted text; limitations stated in the application (Section 5.6.2).

Table 3.4: Cost–benefit comparison

	Lawyer review	Hosted LLM service	This system (local)
Marginal cost per contract	High (professional fees)	Per-request fee	Near zero
Time per contract	Hours to days	Seconds	About 30 seconds
Privacy	Confidential	Contract leaves the user’s machine	Contract stays on the machine
Reliability	Expert judgement	Unmeasured on this task	Measured; misses many clauses
Legal standing	Legal advice	None	None

a long contract that most likely matter and explaining them in plain language, at almost no cost. Expert review in this field is very valuable (the CUAD labels alone are said to represent about two million US dollars of lawyers' time [1]), so even saving part of the time needed to find key clauses is worth something.

Limits of the benefit. The value depends on how reliable the system is. Only 22.0% of real clauses get through the whole system, and the main setup misses 36.9% of High-risk clauses (Section 5.2.2). So the system cannot replace a lawyer. Its value is as a free first look, not as a replacement for professional review.

Chapter 4

Proposed Methodology

4.1 Design Process or Methodology Overview

The design followed the goals and limits from Chapter 3, using a “test before you build” process with four rules: (i) build a strong, simple baseline before any complex model; (ii) measure the best result a design could ever reach before spending training time on it; (iii) fix the train/test split by contract once, use it everywhere, and never tune on the test set; and (iv) test each step on its own and then the whole system from start to finish. The result is a three-step system with a risk layer on top.

The system works in steps (Figure 4.1). The contract text is cut into windows. The presence step decides which of the 41 categories the contract contains. The span step finds the exact clause text for each category found. The risk layer adds the *category’s* High/Medium/Low level, and a text-generation step adds a plain-English sentence. The final output is a report sorted by risk. (The Clausify app runs every step; Section 4.4.9.) Section 5.1.3 shows that the last step does not really summarize the clause in front of it: it picks one of 41 sentences written in advance. So the system is better described as “sorting into categories and showing a template” than as summarizing. Figure 4.1 names the steps and the models behind them. Figure 4.12 at the end of this chapter shows the same steps with the actual data passed between them, which is useful when reading the implementation.

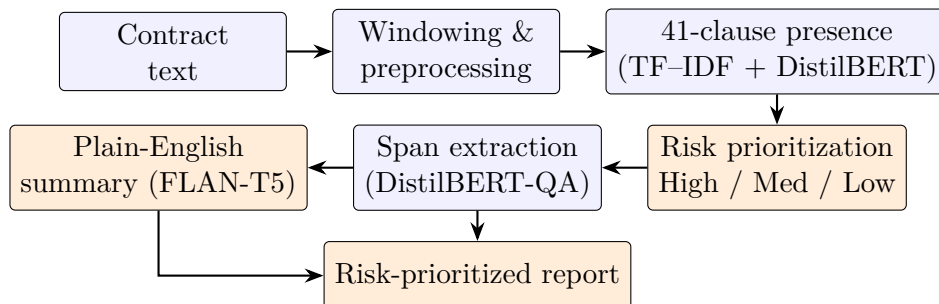


Figure 4.1: System pipeline

One idea runs through this whole chapter: **build a strong, cheap baseline first, and measure the best possible result of a design before spending training time on it.** Both habits changed the design in real ways, as the next sections show.

4.2 Preliminary Design or Design (Model) Specification

For each step we considered other designs and, where possible, tested them before choosing one. For finding which clauses are present, we compared three designs (a simple word-based model that reads the whole contract, a “search first, then classify” transformer, and a transformer that reads the contract in sliding windows) and three ways of combining the simple model with the transformer. We also considered sparse-attention models and rejected them for the reasons in Section 2.1.2; we measured their cost later (Section 4.4.7). The design chosen for each step is described below.

4.2.1 Stage 2A: Presence Classification

A strong baseline first: full-document TF-IDF

Simple bag-of-words models have no limit on input length, so a TF-IDF model can read the *entire* contract, which no windowed transformer can do. We built a TF-IDF vocabulary (20,000 features, single words and word pairs, sublinear TF, `min_df = 2`) from the 408 training contracts and trained one balanced logistic-regression classifier per category. Legal writing is very repetitive (“*This Agreement shall be governed by the laws of the State of ...*”), so the words alone carry a lot of information. This baseline sets the score every neural model has to beat.

Measuring the retrieval ceiling, and dropping retrieval

The obvious way to give a long contract to a transformer is to search first: cut the text into 2,000-character windows that move forward 1,500 characters at a time (the windows are drawn in Figure 4.3), score each window by its TF-IDF similarity to the category, and give the model only the best window or windows. The search uses only information that is available when the app runs (the category name or its CUAD question, never the answer), so no answer information leaks in.

Before training anything, we measured the *search hit-rate*: how often does the chosen window really contain the clause? Figure 4.2 shows how this works and why the result is a hard limit. Whatever this number is, no model after the search can do better, because a clause in a window the search never picks can never be found. The best search we tried (using the category name, keeping the top 3 windows) reached only **64.0%** (Table 4.1). Surprisingly, the CUAD question text did worse than the plain category name, because the questions share so much standard wording that they all look almost the same. A recall limit of 0.64 is well below the baseline’s recall of 0.81 (and its micro-F1 of 0.775), so a search-first transformer would lose before it was even trained. This measurement took seconds and saved us a wasted training run.

This number needs one note. We measured the limit of *word-based* search only: every search in Table 4.1 compares windows by shared words, so a clause written

without the words of its category name cannot be found even when it is clearly there. A meaning-based search (one that turns both the question and each window into vectors) matches meaning instead of shared words and would probably raise the 64.0% figure, possibly above the baseline. We did not test one. So the fair conclusion is not “search does not work”, but that *word-based* search sets a limit that is too low to build on. That was enough to move us to the window design in Section 4.2.1. Whether meaning-based search could raise the limit enough to compete is an open question that we list as future work.

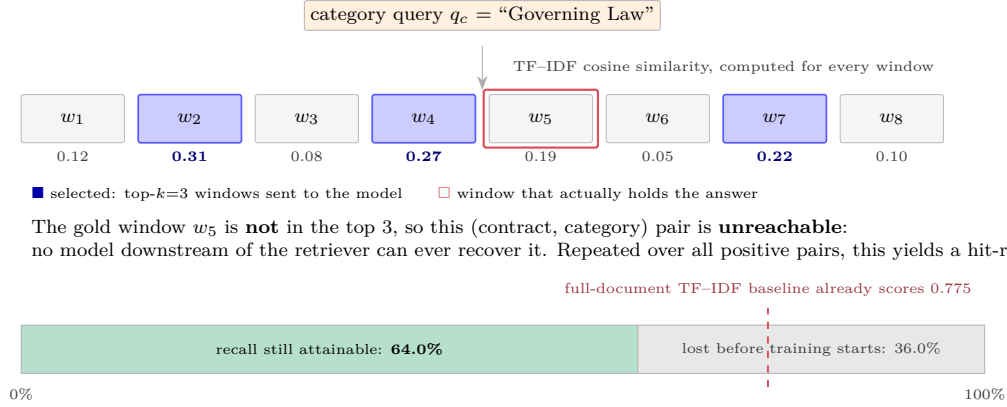


Figure 4.2: Retrieval ceiling

Table 4.1: Retrieval hit-rates

Query strategy	Top- k	Hit-rate
CUAD question text	1	38.6%
Category name	1	44.6%
Category name	3	64.0%
Name + question	3	61.5%

Note. The hit-rate is the recall ceiling a retrieval-based design would impose.

Sliding-window max-pooling (multiple-instance learning)

To remove this limit, we trained a *window-level* classifier instead. Given a pair (category question, window), it predicts whether that window contains a clause of that category. Figure 4.3 shows how the windows are made. A 2,000-character window that moves forward 1,500 characters at a time leaves a 500-character overlap between neighbouring windows. This overlap makes sure no clause can fall between two windows and be lost: the typical (median) clause is 196 characters long (Section 4.3.5), much shorter than the overlap, so every clause fits completely inside at least one window.

The training labels come from the marked character positions: any window that contains a marked clause is positive, and for each (contract, category) pair we also take two random negative windows. Using the marked positions this way is normal supervised learning, not leakage: the labels only decide the training targets, they

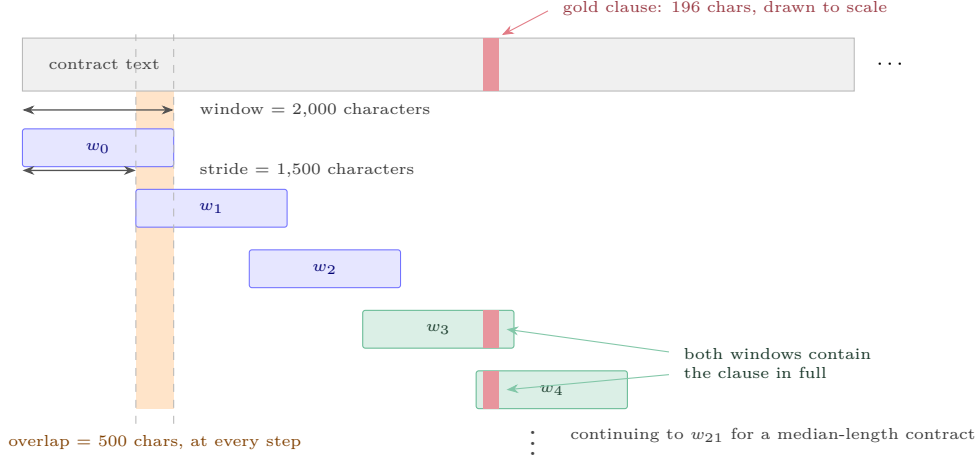


Figure 4.3: Sliding-window generation

are never part of the model’s input, and nothing from the answers is used when the app runs. From the 408 training contracts this gives 53,662 window examples (39.2% positive).

When the app runs, the model scores *every* window of the contract, and the answer for the whole contract is simply the highest window score. This is the multiple-instance learning rule, shown in Figure 4.4. The document is a *bag* of windows; the bag is positive if any window is positive; and taking the max does exactly that. Because the windows cover the whole document with overlap, every clause can be reached, so the search limit disappears. The cost is precision, and Figure 4.4 also shows why: the maximum is taken over every window, so a single wrong window anywhere is enough to mark the whole contract as positive. A typical contract has 22 windows (the median; the mean is 35), so the chance that some window passes the 0.5 threshold by mistake grows with length: long contracts are more likely to give a false alarm than short ones, whatever they actually contain.

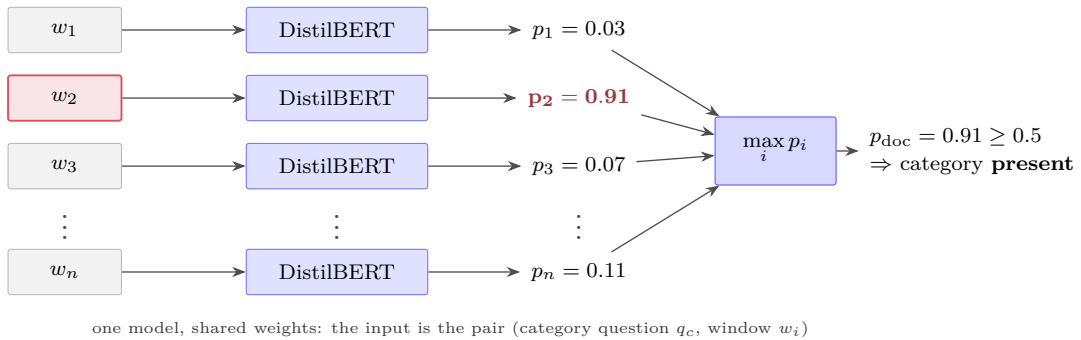
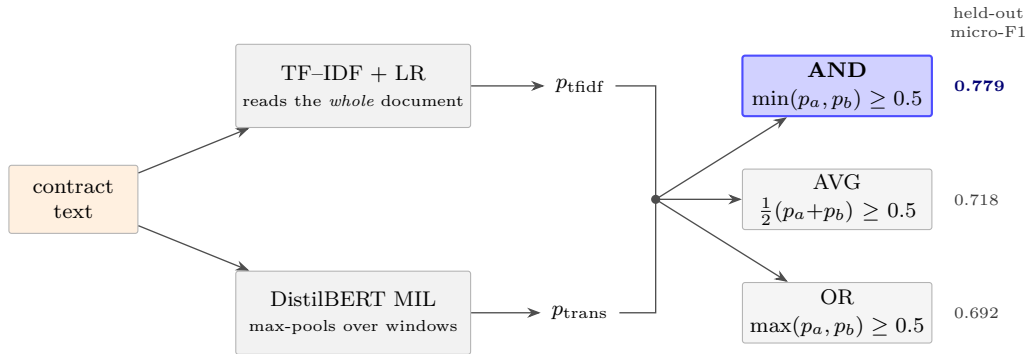


Figure 4.4: Multiple-instance max-pooling

The conjunctive ensemble

When we compared the two models category by category, they turned out to *complement* each other: the transformer does better on categories where meaning matters (for example *No-Solicit of Employees* and *Liquidated Damages*), while the simple

model does better on categories with clear keywords. We combine them with rules that have **no settings to tune** (AND, the min of the two scores; OR, the max; and AVG, the average). We chose such rules on purpose, so that nothing can be fitted to the test set. Figure 4.5 shows the decision. The AND rule, which reports a clause only when both models agree, removes many false alarms: for a wrong detection to survive, both models must make the same mistake on the same contract and category, and their mistakes are mostly independent. This wins back the precision that taking the maximum loses.



All three rules are **parameter-free**: there is nothing to fit, so the test set cannot tune them. AND fires only when *both* models cross the threshold, so a false positive has to be made twice, independently, to survive.

Figure 4.5: Ensemble rules

4.2.2 Stage 2B: Span Extraction

For this step the transformer has no replacement: a bag-of-words model cannot point to positions in the text. We fine-tune **DistilBertForQuestionAnswering**: the input is (CUAD question, window text), and the model predicts where the answer starts and ends. The character positions in `CUAD_v1.json` are turned into token positions using the tokenizer’s offset mapping.

Two different window sizes are used in this project, and it is worth being clear about them, because mixing them up makes the design look inconsistent. The 2,000-character window of Section 4.2.1 is a *presence* window: it covers the document in order, because at that point we do not know where anything is. The window used here is a *focus* window: it is 1,200 characters long, and instead of covering the document in order it is placed around the marked answer, so the answer starts about 150 characters in. Figure 4.6 draws the two at the same scale. The smaller, centred window is needed for a simple technical reason: it makes sure the answer is not cut off by the model’s 320-token limit. With a 2,000-character window, an answer near the end could be cut off, and the model would be trained on an example whose answer is missing. Also note that this step has no length limit at all: it only searches *inside* a window that the presence step has already marked as relevant.

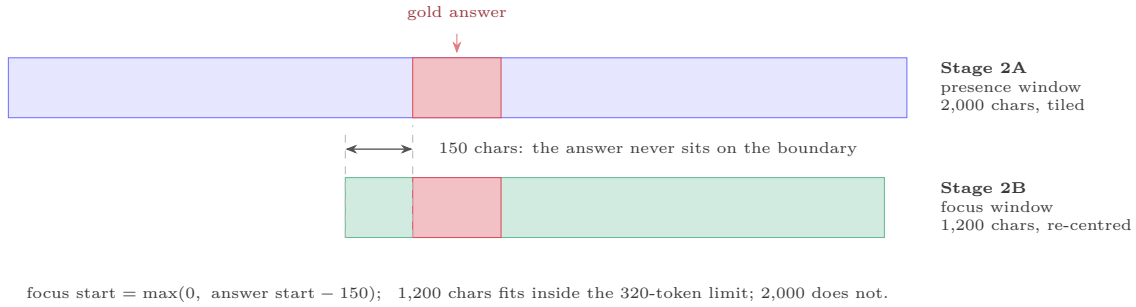


Figure 4.6: Presence vs. focus window

4.2.3 Stage 1: Plain-Language Summarization

CUAD gives us clauses but no plain-English versions to learn from, so we wrote our own: **41 template sentences**, one carefully written one-sentence summary for each category, written from the weaker party’s side. For *Anti-Assignment*, for example, the template is “You cannot transfer this contract to someone else without permission.” Each training clause is paired with its category’s template, which gives 11,156 training pairs, and FLAN-T5-small is fine-tuned on the prompt “**summarize in plain English: <clause>**”.

We test the summarizer with ROUGE-L *and* by comparing, on the same clauses, the fine-tuned model with the original model before fine-tuning. We use both tests on purpose. With only 41 different target sentences, a model can get a high ROUGE score just by recognising the category and copying its template word for word, without really summarizing anything. Chapter 5 shows that this is exactly what happens, and we report it that way.

4.2.4 Category-Level Risk Prioritization

First, a note on the name, because it limits what we can claim. The system does not read a clause and judge how dangerous *that* clause is. It finds which of the 41 categories a contract contains and gives each category a fixed level. These are different tasks, and mixing them up would greatly overstate the system: our layer says “*contracts with a non-compete deserve attention*”, never “*this non-compete is unusually harsh*”. So throughout this report we call it **category-level risk prioritization**, not risk classification.

Assignment criteria

We did not set the levels by gut feeling alone. We state the rules we used so that a reader, or a reviewing lawyer, can disagree with a specific level for a specific reason. Each category was scored on five points, from the weaker party’s side:

1. **Money at risk.** Can this clause cost money, and is there a limit? Having no limit is the strongest single sign of High risk.

2. **Cannot be undone.** If the clause takes effect, can the situation be reversed? Losing ownership of intellectual property or giving a permanent licence cannot be undone later; a notice period can.
3. **Limits on future freedom.** Does the clause limit what the party can do *after* the contract, or outside it, such as who it may work for, sell to or hire?
4. **One-sided.** Does the duty fall on only one side? A shared duty of confidentiality is less dangerous than a one-way indemnity with the same wording.
5. **How long it lasts.** Does the duty continue after the contract ends?

The rule we used: **High** if the category has no money limit (point 1), cannot be undone (point 2), or limits freedom after the contract (point 3). **Low** if the category only gives information (it records a fact such as a date, a name or the governing law, or gives a benefit) and scores on none of the five points. **Medium** otherwise: a real duty, but limited and reversible. This is why *Cap on Liability* is Medium and not Low even though it sounds protective (it limits how much you can recover, which may not cover a real loss), and why *Governing Law* is Low even though it matters in a court case (it is a fact about the contract, not a one-sided duty on the signer).

The eight High categories fall into three groups, each based on a different point: money you could owe without limit (*Uncapped Liability*, *Liquidated Damages*, *Minimum Commitment*, *Most Favored Nation*); something you cannot get back (*IP Ownership Assignment*, *Irrevocable/Perpetual License*); and your freedom to earn later (*Non-Compete*, *Exclusivity*). The full table with reasons is in Appendix A, and Figure 4.7 shows how the categories are spread across the levels.

Who is the “weaker party”?

This phrase is important in the risk table, and it can mean different things, so we explain how we used it. The weaker party is not always the same role: in a distribution contract it may be the distributor, in a licence the licensee, and in a franchise the franchisee. In a balanced contract between equals there may be no weaker party at all.

We defined it as **the party with less power over how the contract was written**, which in CUAD is usually the party that did not write it. Where this was unclear from the category alone, we scored from the side of **the party who carries the duty**: *Non-Compete* is scored from the side that is restricted, *Audit Rights* from the side that is audited, and *License Grant* from the side that receives the licence. This is a rule of thumb, and it has a known weakness: a clause that is High-risk for one side often protects the other side, so the same badge is not equally useful to a licensor and a licensee. A system that judged each clause separately could work out the side from the clause text; ours cannot, so it assumes the user is the side with less power. The app states this assumption in the notice shown above every analysis.

Why a lookup table rather than a learned score

The risk table is a simple lookup, not a model output, and that is on purpose: it can be checked line by line, its reasoning is written down for each category, and it cannot change silently between versions. The cost of this choice is the limit named at the start of this section: it cannot tell a \$10,000 liability limit from a \$10,000,000 one. Section 5.2.2 measures a second cost we did not expect: because the levels are fixed per category, the system’s mistakes depend entirely on which *categories* it finds well, and the High-risk categories turn out to be among the ones it finds worst.

Provenance and validation status of the taxonomy

This risk table is the main thing this project adds to CUAD, and it is also the part with the weakest evidence behind it, so we state exactly where it comes from.

Who wrote it. All 41 levels were set by the three of us. We are computer science students, not lawyers. We set the levels by thinking about the effect on the weaker party (can this clause cost you unlimited money, take away something you cannot get back, or limit what you can do later?) and compared them with the CUAD category descriptions. But they rest only on our own judgement. No legal professional has reviewed them.

Why this matters more here. Every other part of this project is measured against answers marked by experts. This part is not measured against anything. It also affects the user most directly: the level decides what Clausify tells a signer to worry about first. So a mistake here is not just a lower score in a table; it is a warning that never appears. A category we marked Medium that a lawyer would mark High is a silent failure of the app.

The check that is needed, and where it stands. The basic check is a review by someone with legal training: give them the 41 categories with our level and reason, ask whether they agree with each one, and report the number of disagreements *whatever it turns out to be*. We have prepared a review sheet for this (it is in the repository as `review/risk_taxonomy_review_sheet.csv`, with one row per category and columns for the reviewer’s own level and comments), and it takes a reviewer about half an hour.

One reviewer would give a basic check; *two or three reviewing separately* would give a stronger one, and the difference is worth explaining. Simple percentage agreement is a poor measure for this table because the levels are uneven: 22 of the 41 categories are Medium, so a reviewer who answered “Medium” every time would agree 54% of the time while adding no information at all. Agreement corrected for chance (Cohen’s κ for two reviewers, Fleiss’ κ for three or more) removes this problem, and that is what we would report. The repository includes `review/agreement.py`, which reads the completed sheets and prints each reviewer’s agreement, both kappa values and the list of categories in dispute. This way the analysis is fixed in advance and cannot be shaped by the results.

At the time of writing this review has not been done, and no number from it is reported anywhere in this project. Until it is done, the risk table should be read as a documented and checkable, but clearly unchecked, design. Its strength is that every

level is written down with a reason and can be argued with line by line, not that any qualified person has agreed with it yet.

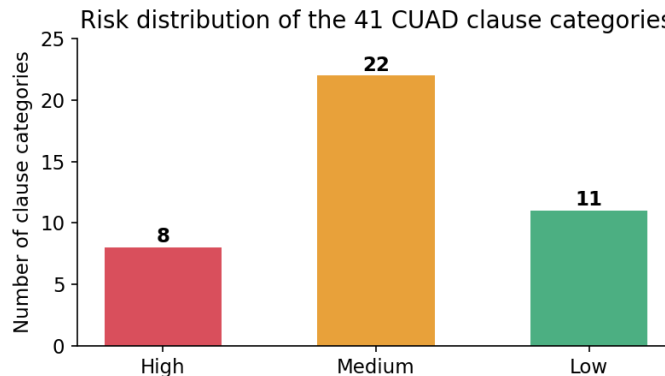


Figure 4.7: Risk-level distribution

4.2.5 Leakage Discipline

With such a small dataset, data leakage is the most dangerous way to fool ourselves, so we followed one clear rule throughout: *the answers may be used to create training targets, but never to shape what a model reads or what is chosen when the app runs*. In practice: the split is by contract; searches use only the category or its question; the marked answer positions label training windows but are never used to build inputs or choose windows when the app runs; and the ensemble rules have no settings, so the test set cannot tune them.

One result of this is visible in Figure 4.6 and must be stated clearly, because it is the one place where the rule seems to be broken. The focus window is placed using the marked answer position, which is part of the answer. In training this is normal supervised learning: it decides which text becomes a training example. In the real system there is no marked answer, and the span step reads the window chosen by the presence step. The *separate* span test in Section 5.1.2 is the exception, and we say so plainly: it builds each test example with the same centred focus window, so it measures extraction *when the position of the answer is already known*. That makes it a best-case score, not a score the real system can reach. The realistic number is the start-to-finish test in Section 5.2.1, which never uses the marked answer position.

4.3 Data Collection

All experiments use the official CUAD release from The Atticus Project, downloaded from the Hugging Face hub ([theatticusproject/cuad](https://huggingface.co/theatticusproject/cuad)). One practical finding is worth noting: the default `load_dataset()` option gives only the 511 original contract PDFs, which cannot be used for training. The marked data is actually stored in two files in the repository, and each one has a different job in our system:

- `master_clauses.csv`: a wide table with 83 columns and one row per contract, which we used to explore the data and build labels;

- **CUAD_v1.json:** the SQuAD-format file with the full contract text and the character positions of every answer, which we used to train the span model and to read whole contracts.

4.3.1 Data Cleaning

The CUAD files were prepared by experts, so cleaning was about fixing format problems rather than wrong values. Four problems had to be handled:

- **Empty cells.** Clause-text cells that are empty (NaN) are treated as an empty list, which means the category is absent.
- **Badly formed lists.** Each clause-text cell holds a Python list written as text. Cells that cannot be read as a list are kept as one clause instead of being thrown away, and blank entries are removed from every list.
- **Inconsistent column names.** The column name `Notice Period To Terminate Renewal- Answer` contains an extra space that breaks matching by name, so columns were paired by their position instead (Section 4.3).
- **Empty answers in the JSON file.** Answers that are empty after removing spaces are ignored when building the span-training examples and the answer lists.

No unusual values were removed: very long contracts are real and were kept, because handling length is exactly what the method must do.

4.3.2 Data Transformation

The CSV is wide: 41 categories $\times$ (a clause-text column plus an answer column), with a file-name column at the front. Turning it into a long table with one row per (contract, category) required two decisions that were not obvious at first:

Pair columns by position, not by name. The column name `Notice Period To Terminate Renewal- Answer` contains an extra space, which silently breaks any matching between a category column and its answer column by name. Pairing by column position (clause text in odd columns, answer in even columns) is not affected by such typos.

Decide presence from the clause-text list, not the Yes/No answer. Each clause-text cell holds a Python list written as text, either `'[]'` or `"['clause', ...]"`. We say a category is *present* when this list is not empty. This rule works the same way for the eight categories (dates, party names) whose answer column holds a value instead of Yes/No, and it also gives us the clause text directly.

This gives $510 \times 41 = 20,910$ labelled (contract, category) rows, of which 32.1% are positive.

Three more steps prepare the data for the models. **Windows:** every contract is cut into 2,000-character windows that move forward 1,500 characters at a time, so neighbouring windows overlap by 500 characters, which is more than a typical clause,

and no clause falls between two windows. **Position conversion:** for the span model, CUAD’s character positions are turned into start and end token positions using the tokenizer’s offset mapping, inside a 1,200-character focus window around each answer so that it fits in 320 tokens. **Turning text into numbers:** for the baseline, each whole contract becomes a TF–IDF vector of single words and word pairs, with sublinear term frequency. Category labels are kept as simple present/absent values; no other scaling of numbers is needed beyond what TF–IDF does.

4.3.3 Data Integration

The two CUAD files complement each other. `master_clauses.csv` gives one row per contract with the clause lists we used to explore the data and build labels, while `CUAD_v1.json` gives the full contract text, the 41 category questions and the character positions of every marked clause. All the modelling uses the JSON file: presence labels, answers and contract texts all come from it, matched by contract title and by the category name taken from each question ID. The CSV is used to explore the data and to double-check how often each category appears. The fixed split by contract is computed once over the 510 contract titles and applied in the same way to every table made from the data.

4.3.4 Data Reduction

We reduced the data only to keep training possible on a laptop and to balance the classes, never to throw away information when the app runs:

- **Vocabulary limit for the baseline.** The TF–IDF vocabulary keeps only the 20,000 most common single words and word pairs that appear in at least two contracts.
- **Fewer negative examples.** For window-level training, every positive window is kept but only two negative windows are taken for each (contract, category) pair, giving 53,662 window examples with 39.2% positives.
- **Training on a sample.** The presence model was trained on a balanced sample of 24,000 windows, the span model on 8,000 of 11,156 examples and the summarizer on 4,000 of 11,156 pairs. Section 5.4.3 tests whether training the presence model on a sample hurt it.

4.3.5 Summary of Preprocessed Data

Four features of the data shaped every design decision that followed:

1. **Very uneven categories.** How often a category appears ranges from *Document Name* at 100% and *Parties* at 99.8% down to *Source Code Escrow* at 2.5% and *Price Restrictions* at 2.9%. The middle (median) category appears in about 23% of contracts.

2. **The rarest categories have almost no examples.** *Source Code Escrow* appears in only 13 contracts in the whole dataset. No model can learn such categories reliably, and their individual scores are mostly noise. We keep this warning in mind throughout the results chapter.
3. **Contracts are long.** The median is 33,143 characters (about 5,000 words) and the maximum is 338,211 characters. The median is about $16\times$ what a transformer can read at once (about 2,000 characters), and the longest is about $170\times$. This is the central modelling problem of the whole project.
4. **Clauses are short.** The median marked clause is 196 characters (90th percentile 587, 99th percentile 1,330). So a clause easily fits inside one window; the hard part is finding it in a very long document.

It is important to say clearly that the 510 contracts are the whole dataset. Marking them reportedly cost about \$2M of lawyers' time, so more cannot easily be added. This has three effects: we must start from pre-trained models; over-fitting is the main danger, so all our conclusions rest on test-set numbers; and the train/test split must be made by contract.

The 80/20 Train/Test Split

As our supervisor asked, we split the dataset **80/20 by contract**: each whole contract goes to one side or the other, so no clause from a training contract can appear in the test set. The split uses a fixed random seed (42) and *all three models use exactly the same split*, so “test” always means the same 102 unseen contracts. Table 4.2 summarizes the split; the almost equal share of positives on both sides shows it is balanced. We saved the two sides as `data/train_80.csv` and `data/test_20.csv` so the split itself can be repeated.

Table 4.2: Train/test split

	Contracts	(Contract, category) rows	Positive rate
Train (80%)	408	16,728	32.0%
Test (20%)	102	4,182	32.2%
Total	510	20,910	32.1%

The overall share of positives in Table 4.2 is almost the same on both sides (32.0% vs. 32.2%), but one pair of numbers can hide a badly uneven split: the two sides can match overall while differing a lot category by category. So Figure 4.8 compares the two sides for all 41 categories at once. The two sides match closely across all categories (Pearson $r = 0.986$, average difference 3.4 percentage points). The clear exception is *Termination For Convenience*, which appears in 47.1% of test contracts but only 33.1% of training contracts, a 14.0-point gap. The other differences are all in the rarest categories, where moving one or two contracts changes the percentage by several points. The split was made once at random and never changed to look better, so we report these gaps instead of hiding them. They are worth remembering in Chapter 5, because a category that is more common in the test set counts more in the micro-averaged score than its training frequency would suggest.

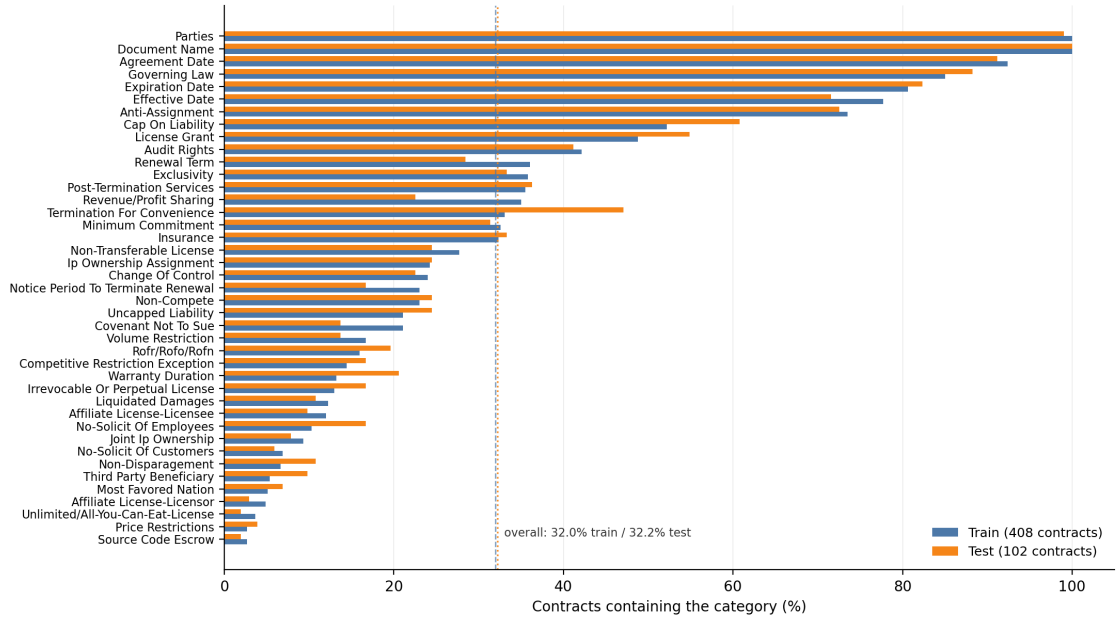


Figure 4.8: Per-category split balance

4.4 Implementation of Selected Design

The chosen design was built in Python as two Jupyter notebooks, `train.ipynb` for preparing data and training and `test.ipynb` for testing on its own, plus some analysis scripts, and it was turned into the Clausify web app. This section describes how each model was trained and how the app works.

All models were trained only on the 408 training contracts (80%). Training ran on one Apple-Silicon laptop using the MPS (Metal) backend with `torch 2.8` and `transformers 5.x`; the summarizer ran on the CPU for the memory reasons explained in Section 4.4.6. Table 4.3 lists the settings.

Table 4.3: Training configurations

	Presence (MIL)	Span (QA)	Summarizer
Backbone	DistilBERT-base	DistilBERT-base	FLAN-T5-small
Task head	Sequence classification	QA (start/end)	Seq2seq LM
Training examples	24,000 (of 53,662)	8,000 (of 11,156)	4,000 (of 11,156)
Max input length	256 tokens	320 tokens	256 in / 48 out
Epochs	2	2	2
Batch size	16	16	8
Learning rate	2×10^{-5}	3×10^{-5}	3×10^{-4}
Device	MPS	MPS	CPU
Wall-clock	≈ 50 – 52 min	≈ 23 – 26 min	≈ 8 – 13 min

4.4.1 Validation Protocol, and Where the Hyperparameters Came From

A reader may fairly ask how the settings in Table 4.3 were chosen. If any of them had been chosen by trying several options and keeping the one that scored best on the 20% test set, every number in Chapter 5 would be too optimistic and the test set would no longer be truly unseen. This section answers that question directly, including one place where we did not do as well as we should have.

What was tuned, and what was not

Nothing in Table 4.3 was tuned. Every setting is either a standard default for the model or a value fixed by reasoning before training, and no setting was ever compared with another on the test set. In detail:

- **Learning rates, batch sizes, epochs** (2×10^{-5} , 3×10^{-5} , 3×10^{-4} ; 16/16/8; 2 epochs) are the standard fine-tuning defaults for DistilBERT and T5 models. We ran each setting once and did no search. This protects the test set, but it also means we have no evidence these are good values, only that they are common ones.
- **Window of 2,000 characters, step of 1,500** comes from a measurement on the *training* data, not from tuning: the 500-character overlap must be longer than a typical clause (196 characters) so that no clause falls between windows (Section 4.2.1). Any window and step that meet this rule would work; we did not look for the best pair.
- **Two negative windows per contract and category** is a choice to balance the classes, keeping about 39% positives instead of letting almost all windows be negative. It was not tuned.
- **Taking the maximum window score** was chosen because it *is* the multiple-instance rule, not because it beat other options; we never tried other options. Section 5.3.1 fixes this gap afterwards.
- **The 0.5 threshold** is simply the default boundary for a two-class model. It was never chosen, checked or changed. Section 5.3.2 shows afterwards that it is a poor choice for the window-based transformer.

In short, this project avoided tuning on the test set by not tuning at all. That is a real guarantee that the test scores are fair, but it is also a limit: the transformer may lose to the baseline partly because nobody looked for better settings for it. We note both sides.

The validation sets actually used

Table 4.4 shows, for each model, what data was kept aside during training and what it was used for.

Table 4.4: Validation arrangements

Model	Validation data	Use
Presence (MIL)	5% of the <i>training</i> windows, split off before training	Monitoring only. Clean: no test contract contributes.
Span (QA)	None (<code>eval_strategy="no"</code>)	No validation loss exists for this model.
Summarizer (deployed checkpoint)	None	The checkpoint used in every reported result and in Clausify was trained with no evaluation set.

Note. “Monitoring only” means the loss was printed and plotted but never used to select a checkpoint, stop early, or change a setting: all three runs used `save_strategy="no"` and kept the final-step weights.

One thing we got wrong

The first exploratory run of the summarizer in the notebook, whose loss curve was shown in earlier drafts of this report, used 300 clauses *taken from the test split* to monitor training. That is the wrong data to monitor with, and we should not have done it.

Two things limit the damage, and we do not use them as excuses. First, that run kept no checkpoints (`save_strategy="no"`) and had no early stopping, so the numbers it printed could not have been used to choose a model; nothing about the model depended on them. Second, the summarizer used for all results was trained again by `build_artifacts.py` with *no* evaluation set at all, and that model, not the notebook’s, produced every summarizer result in Chapter 5. So no reported score comes from a model that was chosen using test data.

What remained was that the “validation” loss printed by that run ($0.096 \rightarrow 0.075$) was really a loss on test clauses, so it should be ignored. The October 2026 re-training rebuilt the summarizer exactly as it is used, with no evaluation set, and Figure 4.11 now shows that run’s training loss, so no test data appears in it. We keep this mistake in the report because it is a useful lesson: with a small dataset and three models, it is easy to grab “the other split” when a validation set is needed. The way to catch it is to write down, for each model, exactly what was kept aside, which is what Table 4.4 now does.

4.4.2 Optimization Schedule

Table 4.3 gives the highest learning rate of each model, but that alone does not describe a training run; the schedule that lowers it over time does. All three models use the same optimizer setup: AdamW with a **linear decay** schedule, **no warm-up**, and gradient clipping at 1.0. Figure 4.9 plots the three schedules. The presence and span settings were read from the `training_args.bin` file saved next to each model, not from our notes, so the plot shows what actually ran.

Two of these choices need a reason. Skipping warm-up is usually risky, but these are short fine-tuning runs (1,000–3,000 steps) that start from pre-trained weights with small learning rates. The usual reason for warm-up, keeping the first updates of a randomly started model stable, hardly applies here, and the smooth loss curves in Figures 4.10 and 4.11 show no early problems. Gradient clipping at 1.0 was kept on throughout, which matters more than it may seem on the MPS backend: it stops a single bad batch from causing the kind of breakdown we saw with DeBERTa-v3 (Section 4.4.6).

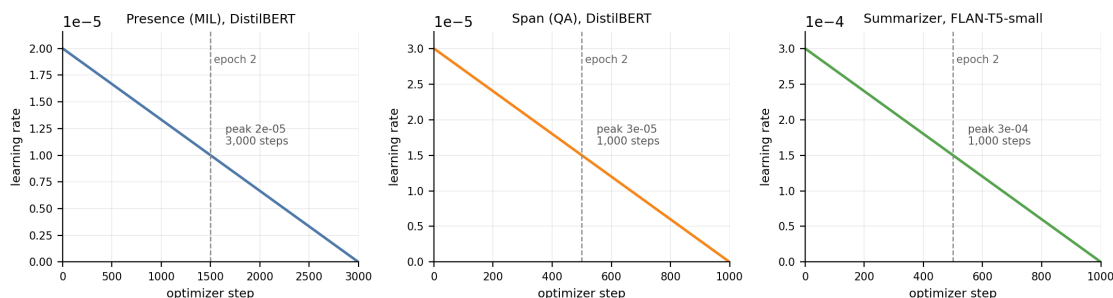


Figure 4.9: Learning-rate schedules

We should be open about one limitation and how far we could fix it. A record of the gradient size at every step is the natural partner of these curves, and it is the one check that shows directly that training stayed healthy, instead of guessing it from the loss. We did not keep it for the three original runs: the temporary checkpoint folders were deleted after training, so only the final models and the loss values below remain from them.

We did keep it for the full-data presence run in Section 5.4.3, which uses exactly the same optimizer setup, and it shows something the loss curves do not. Across the 63 logged points, the gradient size ranges from 0.55 to 11.97 with a median of 4.66, and **98.4% of logged steps are above the clipping limit of 1.0**. Nothing broke (there are no NaN values, no huge spikes, and the loss falls smoothly throughout), so the runs were healthy in the way that matters. But clipping here is not the rare safety net the name suggests. It is active on almost every step, which means the update size is being cut back all the time, so the learning rate of 2×10^{-5} overstates the real step size. We mention this because it slightly weakens the warm-up argument above: part of what kept the early updates stable was almost certainly the clipping, not only the small learning rate. Readers should not take “no warm-up was fine” as general advice based only on our smooth loss curves. Re-running the span and summarizer training with the same logging would close this gap in about half an hour.

4.4.3 Training the Presence Classifier

The TF-IDF + logistic-regression baseline trains in under a minute. On the 20% test set it reaches micro-F1 0.775, weighted-F1 0.777, macro-F1 0.572 over all 41 categories, and 0.666 over the 34 categories that have at least ten test positives. The difference between micro and macro scores comes from the rare categories with very few examples, not from a weakness of the model.

The window-level DistilBERT was trained on a balanced sample of 24,000 window examples (keeping 39.2% positives) so that training stays manageable on a laptop. In the October 2026 re-training it took 49.8 minutes. The training loss fell smoothly from 0.57 after the first 100 steps to 0.26 at the end, and the validation loss went from 0.275 after the first epoch to 0.254 after the second (Figure 4.10). The validation loss stayed close to the training loss, so we saw no sign of over-fitting.

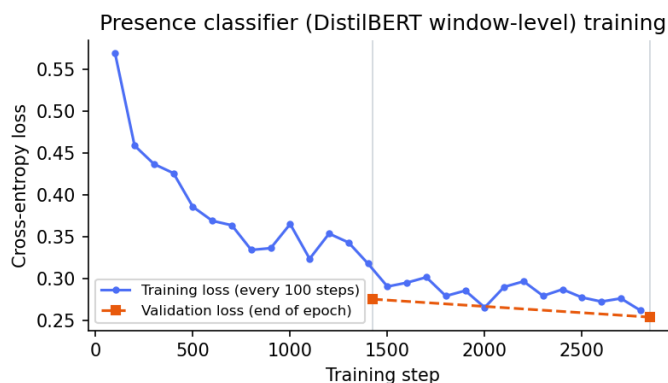


Figure 4.10: Presence classifier loss

4.4.4 Training the Span Extractor

The question-answering model was trained on 8,000 focus-window examples for two epochs. Before training, we turn the labelled token positions back into text for a few examples and check that they match the marked answers. In SQuAD-style systems it is very easy to get the character-to-token conversion wrong without noticing, and this small check catches that.

4.4.5 Training the Summarizer

FLAN-T5-small was fine-tuned on 4,000 clause-template pairs on the CPU: about 8 minutes originally, and 12.7 minutes in the October 2026 re-training (which ran while the laptop was busy with other jobs). The training loss dropped quickly, from 1.17 at step 100 to 0.08 by the end of the second epoch (Figure 4.11). There is no validation curve because the model used in the app is trained with no evaluation set (Table 4.4). The re-trained model scores ROUGE-L 0.775 against the templates on the 150 test clauses, the same as the original. Looking back, this fast drop was the first sign of the template-copying behaviour we analyse in Chapter 5: with only 41 different targets, the task is simply easy to learn.

4.4.6 Engineering Notes: Training on Consumer Hardware

A few practical lessons may save time for anyone repeating this work on Apple-Silicon computers:

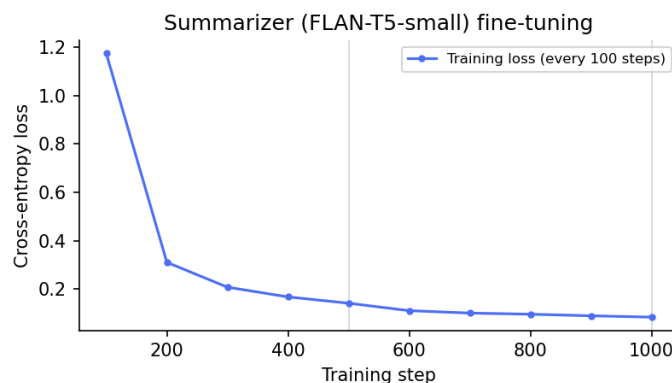


Figure 4.11: Summarizer loss

- **DeBERTa-v3 is unstable on MPS.** Our first-choice model produced NaN (broken) gradients on the MPS backend and was far too slow on the CPU (about 89 seconds per step). We switched to DistilBERT; the method stays the same, and on a CUDA machine the model name can simply be changed back.
- **MPS memory is not freed inside one program.** After two transformer trainings in the same session, the summarizer ran out of MPS memory even after we deleted objects and cleared the cache. What actually works is restarting the kernel or training the (small) summarizer on the CPU, which is what we did.
- **Library changes.** `transformers 5.x` renamed `Trainer(tokenizer=...)` to `processing_class=...`. We also turned off mixed precision (`fp16=bf16=False`) and converted the loss logits to fp32 to avoid data-type problems on MPS.
- **Repeatability.** Everything needed (the split in `artifacts/split.json`, the baseline in `artifacts/baseline.pkl`, and the three trained models) is saved to disk, and a separate test notebook recreates every number in Chapter 5 from those files alone.

4.4.7 Measured: What Sparse Attention Would Have Cost

Section 2.1.2 argued against sparse attention partly because of cost, and a cost argument is weak if the cost is never measured. The laptop used for this project is exactly the machine in question, so measuring it only cost the time to run it, and we did.

We built Longformer-base with its published size (12 layers, 768 hidden units, 4,098 position embeddings, attention window 512, 148.7M parameters). We timed one forward pass at 4,096 tokens and one full training step (forward pass, backward pass and AdamW update), and compared them with the DistilBERT setup this project actually used. The weights are random, which is fine here, because time and memory depend on the size of the tensors, not on the values in them. Each setup ran in its own program, so the peak memory reported belongs to that setup alone. The machine is an Apple M4 with 16 GB of shared memory, which the operating

system also uses. Table 4.5 shows the results.

Table 4.5: Longformer vs. DistilBERT cost

Model	Operation	Batch	Time	Peak memory
Longformer-base (148.7M)	forward, 4,096 tok	1	3.69 s	2.18 GB
DistilBERT (67.0M)	forward, 256 tok	16	0.19 s	1.15 GB
Longformer-base	training step, 4,096 tok	1	258 s	14.71 GB
Longformer-base	training step, 4,096 tok	2	2,170 s	20.75 GB
DistilBERT	training step, 256 tok	16	3.17 s	4.47 GB

Note. Measured on the Apple M4 (16 GB unified memory) that trained every model reported in this project.

First, this measurement corrects us. The original wording of Section 2.1.2 said that fine-tuning a Longformer at 4,096 tokens was “far beyond a single laptop’s memory budget”. That is not what happens. At batch size 1 it fits: 14.71 GB on a 16 GB machine. That is very close to the limit, leaving less than 1.5 GB for the operating system, but the model, its gradients and its optimizer state all fit in memory and the step finishes. We had claimed it was impossible, it turns out to be a tight fit, and our claim was wrong.

The memory claim is true one step further. At batch size 2 the memory needed rises to 20.75 GB, more than the machine has, and the effect shows in the timing: the step takes 2,170 seconds instead of the roughly 516 that doubling the batch should take. That is 8.4 times slower for twice the work, which is the sign of the computer swapping memory to disk instead of computing. So the honest claim is not “it does not fit” but “it fits only at batch size 1, and any batch size a real training run would use does not fit”.

The real limit, however, is time, not memory. At 258 seconds per step and batch size 1, repeating our two-epoch presence training (22,800 training windows, 45,600 examples seen in total) would take about **136 days** of non-stop computing. The DistilBERT run it would replace took about 50 minutes. Even if we allow for extra start-up time in a short test (our own DistilBERT step takes 3.17 s here but about 1.0 s during the real run, so these numbers may be about three times too high), the conclusion does not change: a third of 136 days is still 45 days, and the project had weeks, not months.

The cost when the app runs is also confirmed by measurement. Scanning one typical contract (33,143 characters) for all 41 categories takes $22 \text{ windows} \times 41 \times 11.7 \text{ ms} \approx \mathbf{10.6 \text{ seconds}}$ with our DistilBERT, compared with $3 \text{ windows} \times 41 \times 3.69 \text{ s} \approx \mathbf{454 \text{ seconds}}$ (seven and a half minutes) with Longformer. Clausify is meant to respond within seconds, and this measurement rules Longformer out for the app, whatever a large training computer could do.

None of this shows that a Longformer would have scored worse. It shows only that we could not have found out on this hardware, which is a much narrower claim than the one we started with. Settling the modelling question needs a CUDA machine, and we list it as future work instead of pretending the cost argument answers it.

4.4.8 End-to-End Data Flow

Figure 4.1 names the steps; Figure 4.12 adds what actually passes between them. The sizes shown are for one contract of typical length passing through the system.

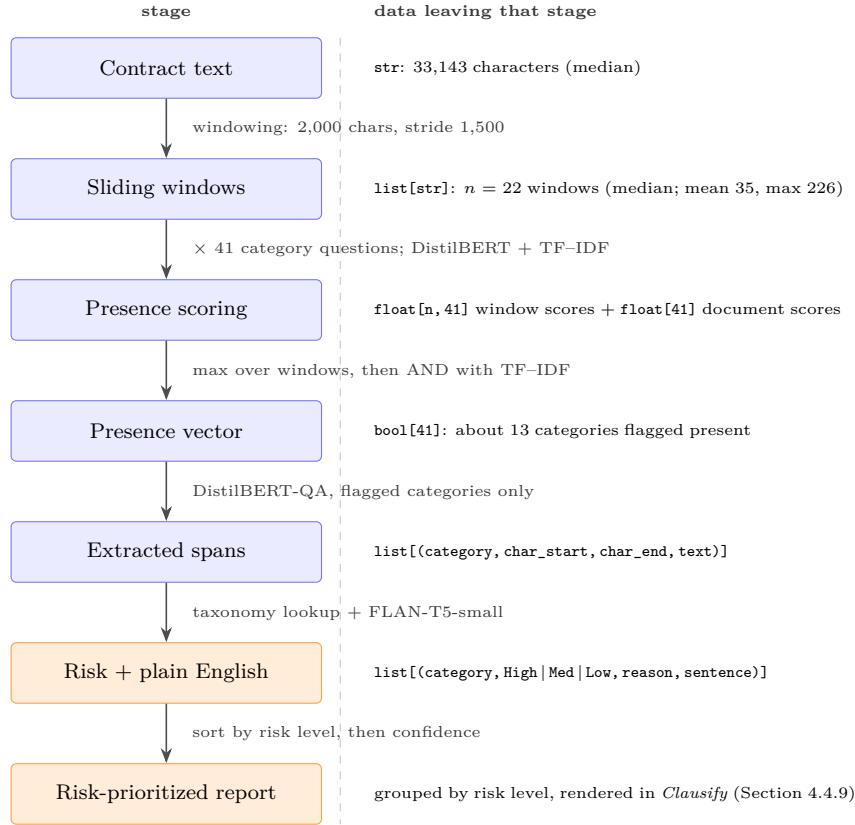


Figure 4.12: Pipeline data flow

4.4.9 The Clausify Application

Purpose and Design

The final result of the project is *Clausify*, a Streamlit web app that runs all of the project’s trained models on any contract the user gives it (the DistilBERT presence classifier, the TF-IDF baseline for the optional AND-ensemble, the span model and the FLAN-T5 summarizer) and shows the results as a report sorted by risk. It is the system of Figure 4.1, made easy to use. The design goal is simple: a non-lawyer should see the “watch out before signing” items first. Clauses found are grouped **High** → **Medium** → **Low** and sorted by model score within each group.

User Workflow

1. **Add a contract.** Paste the text, or upload a .pdf, .docx or .txt file; the text is read on the user’s own computer (Figure 4.13). The user picks *Whole contract*, *Separate clauses* (Section 4.4.9) or *Auto*. A settings panel chooses the detection rule (DistilBERT alone, the default, or the AND-ensemble) and

turns the plain-English lines on or off. Detection uses the fixed 0.5 threshold used in all our tests.

2. **Analyze.** One click runs all the models; a progress bar follows the scan of the 41 categories and then the summarizer.
3. **Read the report.** A summary bar counts the clauses found at each risk level (Figure 4.15), and below it each clause appears as a card. On the made-up test contract the bar shows 7 High, 14 Medium and 9 Low risk clauses: 30 of the 41 categories found (October 2026 models, DistilBERT detection).

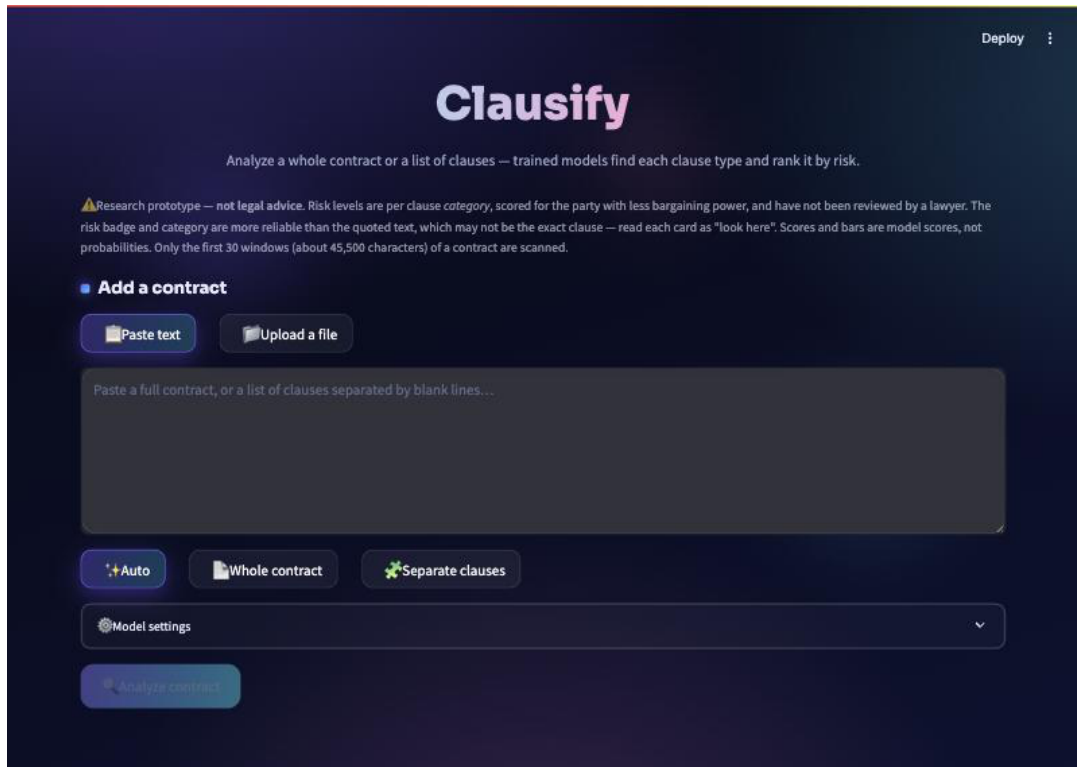


Figure 4.13: Clausify input screen

Each clause card shows the category name with a coloured risk badge; the one-line reason why that category is risky (from the risk table in Appendix A); the summarizer’s plain-English sentence, with a warning when it belongs to another category; the detection score as a bar, with both the DistilBERT and TF-IDF scores; and the clause quoted from the user’s own contract, with the span model’s key phrase highlighted (Figure 4.16). Below the cards the app adds three more views:

- **Highlighted contract.** The full text, with every clause found marked in its risk colour and labelled with its category; each card links to its place in the text.
- **Missing-protection checklist.** Protective clauses the models did *not* find, namely a limit on liability, termination for convenience, governing law, an expiry date, a warranty period, insurance and, when the contract renews itself, a notice period to stop the renewal. Each comes with a sentence on why its

absence matters. The view says that “not detected” can also mean the model missed it.

- **Downloadable report.** A PDF report (summary, every clause with its reason, plain-English line, scores and quoted text, and the checklist) and a CSV file with the same rows.

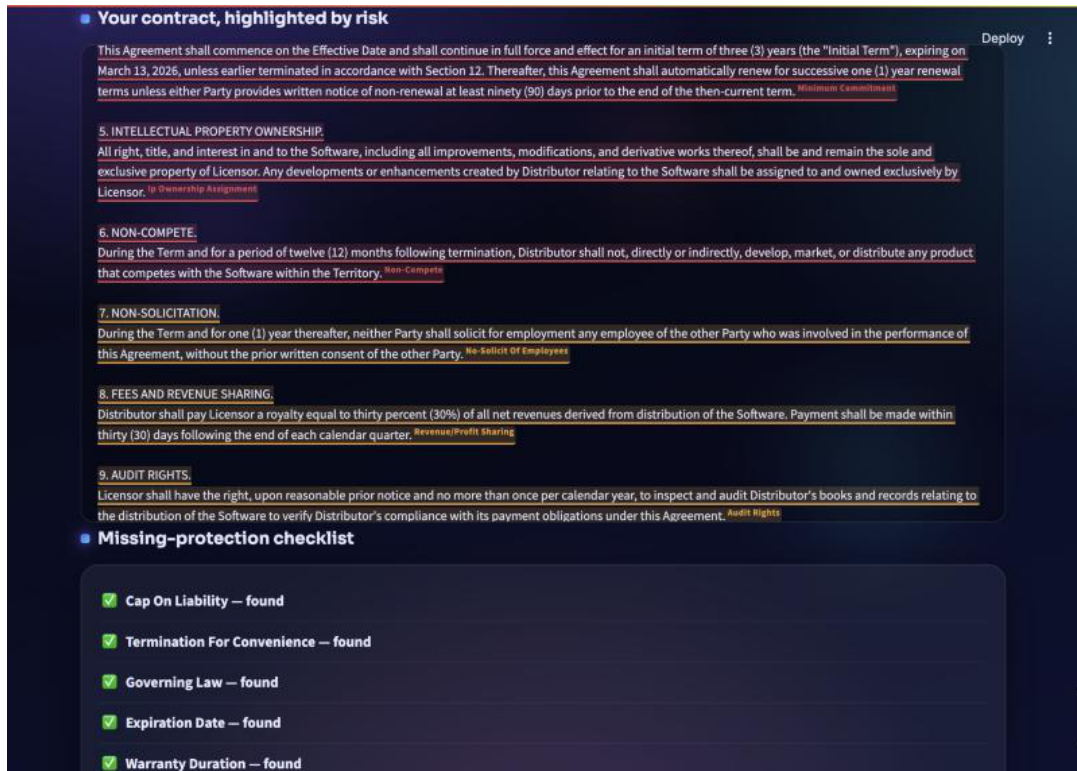


Figure 4.14: Highlighted contract and checklist

Figure 4.14 shows the highlighted view and the checklist on the made-up test contract. The renewal paragraph labelled *Minimum Commitment* there is a real mistake by the model, which we kept on purpose instead of choosing a cleaner example.

How the Models Run Inside the App

The app follows exactly the same steps that were used in testing:

1. **Windows.** The contract is cut into 2,000-character windows that move forward 1,500 characters at a time, the same as in training. At most 30 windows are read, so very long documents stay fast.
2. **Model 1: presence.** For each of the 41 categories, the exact CUAD question the model was trained on is paired with every window and scored by the window-level DistilBERT. The score for the whole contract is the highest window score; categories above the threshold are reported as present, and the best window is remembered.
3. **Finding the clause.** Inside that best window, the presence model scores each paragraph against the category question, and the highest-scoring paragraph is

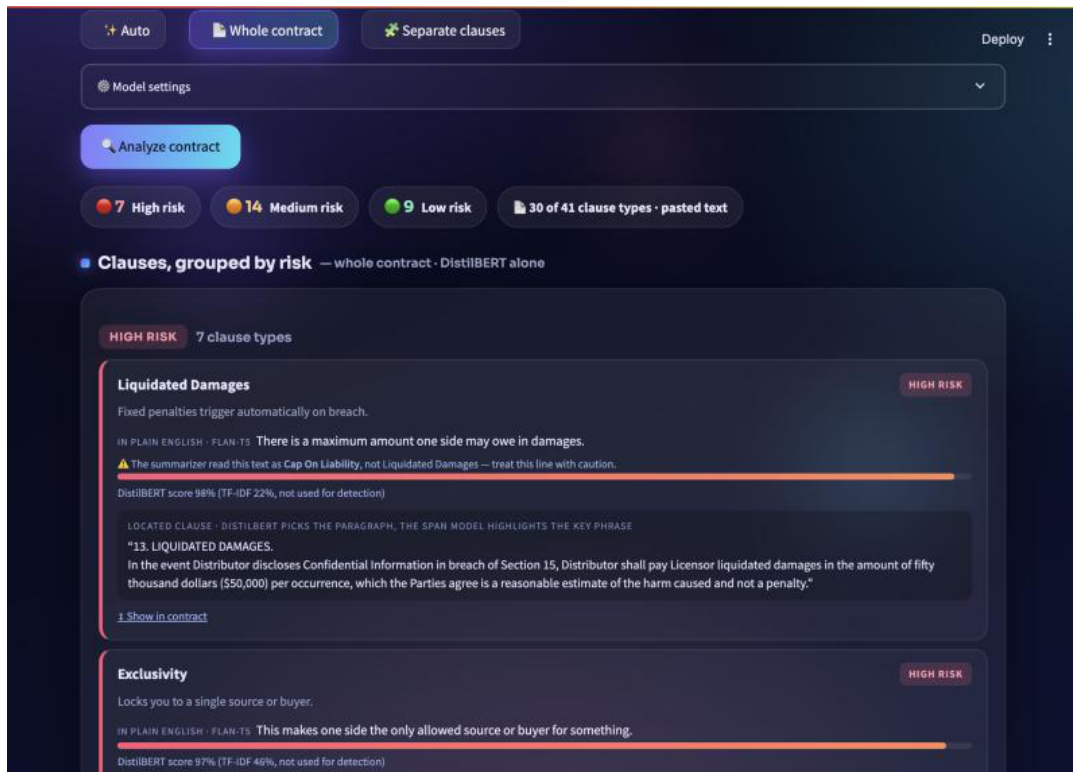


Figure 4.15: Clausify results view

quoted on the card; the DistilBERT question-answering model then highlights the key phrase inside it. An earlier version let the span model choose the text from the whole window, and on test inputs it returned the same sentence for almost every category: the span model prefers text near the position where answers were during training (Section 5.2.1), so it is now used only inside the paragraph the presence model chose.

4. **Optional AND-ensemble.** With this setting, the TF-IDF baseline scores the whole contract, and a category is reported only when both models' scores reach 0.5, exactly as tested in Section 5.4.1.
5. **Risk layer.** The category found is looked up in the fixed risk table (8 High, 22 Medium, 11 Low) for its level and reason.
6. **Summarizer.** FLAN-T5-small receives each quoted clause with the same prompt and settings as in testing and returns one plain-English sentence.

All the models run on the CPU, so the app works on any computer; a typical contract takes about 27 seconds once the models are loaded (Table 4.6). The app loads the actual trained models saved by the training code (`presence_mil`, `span`, `summarizer` and the `TF-IDF baseline.pkl`). When they are not on the computer, as on a cloud server, it downloads the same files once from a public Hugging Face model repository.

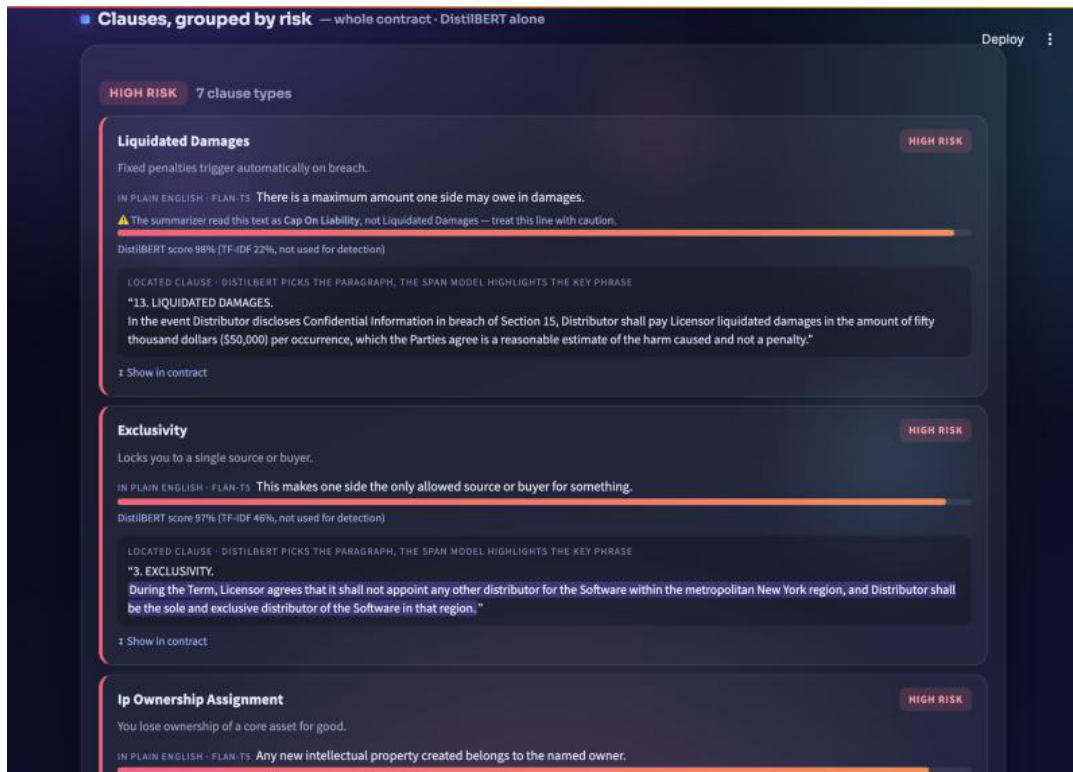


Figure 4.16: High-risk clause cards

Implementation Notes

The app is a single-page Streamlit program (`app.py`) built on two small modules: `model_utils.py`, which loads the models and handles windows, scoring, clause finding, the summarizer and the risk table, and `features.py`, which reads uploaded files, builds the highlighted view and creates the reports. Two test contracts come with the app: a real CUAD contract, and a made-up “Master Software License and Distribution Agreement” that we wrote to cover many of the 41 categories. On that made-up contract the app finds clauses at all three risk levels and puts High-risk clauses such as *Liquidated Damages*, *Exclusivity*, *IP Ownership Assignment* and *Uncapped Liability* at the top of the report: exactly the before-you-sign warnings the design aims for.

Clause-by-Clause Mode

Users often paste a few clauses instead of a whole contract and expect one card per clause. Whole-contract mode does not do this: for each of the 41 categories it asks whether that category appears anywhere in the window, so six related clauses pasted together gave fourteen categories, eight of them wrong. So the app has a second mode (chosen automatically for a short list of paragraphs, or chosen by the user) in which each paragraph is one clause and gets exactly one category and one risk level. When most paragraphs are numbered (“1.”, “4.2”, “(a)”, “Section 5”), only the numbered ones are treated as clauses; the title, opening paragraph and signature lines are listed separately as skipped, so a contract with 18 numbered clauses gives exactly 18 cards, numbered as in the contract.

In this mode, if a clause heading names a category (“2. EXCLUSIVITY.”), that heading decides the category directly; headings that do not clearly name one category are ignored. Otherwise the presence model scores the clause against all 41 category questions. The raw scores cannot simply be compared across categories (*Parties*, for example, scores about 0.98 on almost any clause that names the two sides). So each category’s score is adjusted by how that category scores on real CUAD clauses of *other* categories, using 1,586 clauses from the training split, and the clause gets the category with the highest adjusted score. A clause whose best adjusted score is below 2 is shown as *unrecognized* instead of being forced into a category.

Tested on 721 clauses from the *test* split, this adjustment raises the share of clauses given the right category from 9.6% (raw highest score) to **42.3%** out of 41 categories. The right category is in the top three 65.3% of the time, and the right risk level is given 67.0% of the time. This is well below the whole-contract scores, which is expected: the presence model was trained to scan 2,000-character windows, not to label single clauses. The app shows these numbers under every clause-mode result. On the six-clause example above, the version with headings is labelled correctly (five High, one Low). With the headings removed, the model alone gets none of the six risk levels right (it labels five of them Medium and leaves one unrecognized). So for inputs like this, the clause headings provide the accuracy, and cards decided by the model alone should be checked against the clause text.

Measured Latency

Chapter 2 explained that we chose a small model partly because the app must respond quickly. Section 4.4.7 measured what the alternative would have cost; this section measures how fast our app *actually* is, running the app’s own code on test contracts of different lengths.

Table 4.6: Application latency

Contract	Characters	Windows scored	Time	Peak RSS
Shortest in test set	1,283	1	2.1 s	1.06 GB
25th percentile	18,104	12	13.3 s	1.19 GB
Median	35,136	24	27.1 s	1.50 GB
75th percentile	65,547	30 (capped)	32.4 s	1.51 GB
Longest in test set	289,615	30 (capped)	32.0 s	1.52 GB
<i>median 27.1 s mean 21.4 s worst case 32.4 s</i>				

Note. Measured through the application’s own `analyze()` entry point. CPU only, Apple M4, models already loaded (cold load adds 0.4 s). Peak RSS is the whole Python process, both models resident.

An earlier draft said a contract takes “roughly 5–15 seconds”. That was a guess from watching the demo on short inputs, not a measurement, and it was wrong: a typical contract takes 27.1 seconds.

Two design effects show in the numbers. First, the time grows in proportion to the number of windows, because most of the work is 41 categories $\times$ windows passes

through DistilBERT: the model scores each (category, window) pair once and nothing is reused between categories. Second, the 30-window limit keeps the worst case short: without it the longest test contract would take several minutes, but with it every contract longer than about 45,500 characters is cut at the same point. This limit trades speed for a hidden cost in recall: clauses after the cut cannot be found at all, and this is not reflected in any Chapter 5 number, because those tests read the full document. It is a limit of the app rather than of the method, and Section 5.6.2 says so.

Chapter 5

Result Analysis

5.1 Performance Evaluation

What we measure. Each step is measured with the score that fits its task. *Finding which clauses are present* is a yes/no decision for each of the 4,182 (contract, category) test pairs. We score it with micro-F1 (every decision counts the same), macro-F1 (every category counts the same), and the accuracy, precision, recall and specificity from the confusion matrix. *Finding the clause text* is scored with token-level F1, exact match, and the share of answers that overlap the real clause with $F1 \geq 0.5$. *Summarizing* is scored with ROUGE-L against two different sets of reference sentences. The *whole system* is scored by the share of real clauses that get through every step, and the risk layer by how many clauses it finds at each risk level. Speed is measured as the time and memory the app needs (Table 4.6).

How we test. Every model is tested once on the same 102 test contracts, using a separate notebook that loads the saved models from disk. Uncertainty is measured with a bootstrap over contracts, and we check whether the main result changes with the amount of training data, the random seed, the way window scores are combined, the decision threshold and the train/test split.

Everything in this chapter is measured on the 20% test set: the same 102 unseen contracts for every model. All numbers come from the October 2026 re-training (Chapter 4) unless a run is marked as from August 2026. They were produced by a separate test notebook that loads the trained models from disk, so nothing left over from training can affect them.

5.1.1 Presence Classification

Model comparison

Table 5.1 and Figure 5.1 compare all the presence models on the 4,182 (contract, category) test decisions. Two things stand out. First, the window-based transformer *loses on its own* to the simple TF-IDF model that reads the whole document (0.692 vs. 0.775). Legal wording is distinctive enough that word evidence from the whole document beats meaning-based evidence from small windows, and taking the max-

imum window score creates too many false alarms, just as Section 4.2.1 predicted. Second, the AND-ensemble, which reports a clause only when *both* models agree, removes many of each model’s false alarms and comes out on top (0.779). The lead over the simple model is small on 102 test contracts, so we do not rely on it: Section 5.4.1 measures exactly how small it is and finds that this gap cannot be told apart from zero. The conclusion we stand behind is about method: *simple word-based models are strong at deciding which clauses a contract contains, and the transformer is useful as a filter that removes false alarms, not as a replacement.*

Table 5.1: Presence model comparison

Model	micro-F1
Ensemble AND (both must agree)	0.779
TF-IDF baseline (full document)	0.775
Ensemble AVG (mean probability)	0.718
Transformer (max-pool) alone	0.692
Ensemble OR (either fires)	0.692
<i>August 2026 control run, Section 5.4.3:</i>	
Transformer trained on all 53,662 windows	0.694
Ensemble AND on that transformer	0.782

Note. micro-F1 over all 4,182 test decisions. The transformer (0.6924) is just ahead of OR (0.6916).

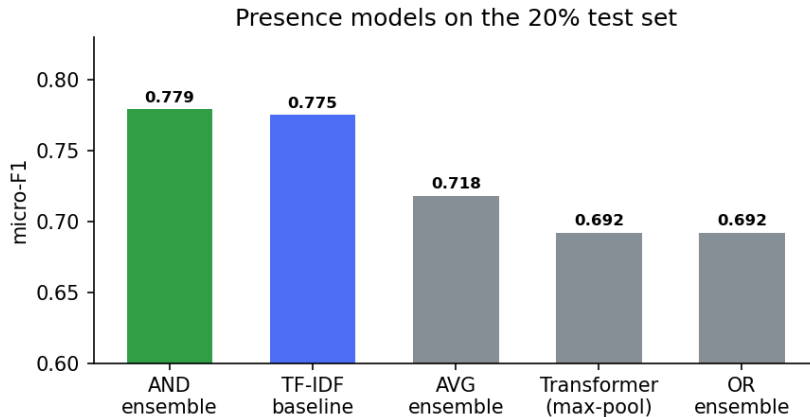


Figure 5.1: Presence model micro-F1

Confusion-matrix analysis

Our test plan required a full confusion-matrix analysis of the final model. Table 5.2 gives the matrix for the AND-ensemble over all test decisions, Figure 5.2 shows the same matrix as a heatmap, and Table 5.3 lists every score taken from it.

Precision and recall are balanced (77.4% and 78.4%), which shows the AND rule did its job: taking the maximum window score alone loses precision, and requiring both models to agree wins it back without losing recall. The specificity of 89.1% means

Table 5.2: AND-ensemble confusion matrix

	Predicted Absent	Predicted Present
Actually Absent	TN = 2,525	FP = 309
Actually Present	FN = 291	TP = 1,057

Table 5.3: Confusion-matrix metrics

Metric	Formula	Value
Accuracy	$(TP + TN)/\text{all}$	85.65%
Precision	$TP/(TP + FP)$	77.38%
Recall (Sensitivity)	$TP/(TP + FN)$	78.41%
Specificity	$TN/(TN + FP)$	89.10%
F1 score	$2PR/(P + R)$	0.7789

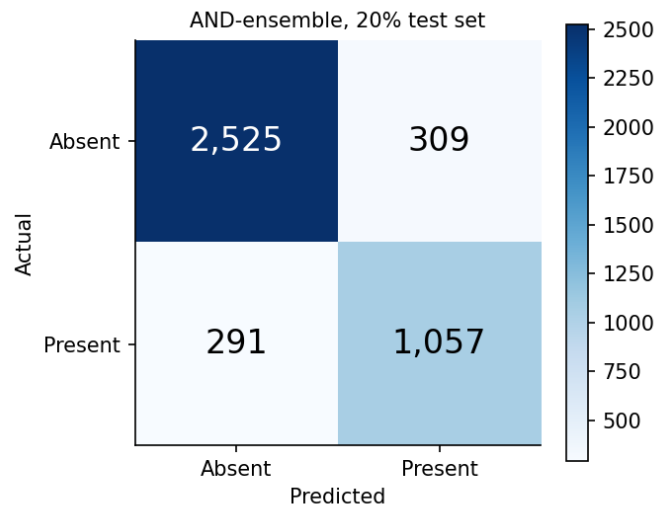


Figure 5.2: Confusion matrix heatmap

the system rarely claims a clause that is not there; for a legal-review tool, that is the more harmful kind of mistake.

Per-category behaviour

Figure 5.3 shows the per-category F1 for the strongest categories. The common, standard categories are almost solved (*Document Name* 1.000, *Parties* 0.995, *Agreement Date* 0.963, *Governing Law* 0.931). At the other end, five categories with at most seven test examples score an F1 of 0.000 with the AND-ensemble. Table 5.4 shows exactly what this means. In every case the ensemble marked the category as present in *no* test contract (no true and no false positives), so F1 = 0.000 is correct: none of the 2–7 real clauses was found. But precision is undefined (0/0) rather than zero. The transformer on its own does find some of these clauses; the AND rule removes them because the TF-IDF model never agrees (Section 5.2.4). With only two to seven examples per category, these scores are not reliable either way. This weakness comes from the dataset, not from any particular model, and it is why our main results use micro-averaged rather than macro-averaged scores.

Table 5.4: Categories with zero F1

Category	Test examples	AND TP / FN	AND precision	Transformer alone F1 (TP/FP/FN)	TF-IDF F1
Most Favored Nation	7	0 / 7	n/a	0.000 (0/0/7)	0.000
No-Solicit of Customers	6	0 / 6	n/a	0.323 (5/20/1)	0.000
Price Restrictions	4	0 / 4	n/a	0.087 (1/18/3)	0.000
Source Code Escrow	2	0 / 2	n/a	0.000 (0/0/2)	0.000
Unlimited/All-You-Can-Eat License	2	0 / 2	n/a	0.500 (1/1/1)	0.000

Note. n/a: the model never said “present”, so precision is 0/0 and has no value. Threshold 0.5, October 2026 models.

5.1.2 Span Extraction

We tested the question-answering model on all 2,667 test windows that contain an answer (Table 5.5). A token-F1 of 0.764, with 82.8% of answers overlapping the real clause at $F1 \geq 0.5$, means that when the clause is in front of the model, it captures most of the right text about eight times out of ten. The lower exact-match score did not surprise us: marked legal clauses are long and their exact start and end are often debatable (should the section number at the start be included?), so overlap is the more useful score here. For comparison, the best published CUAD result uses DeBERTa-xlarge, which has about ten times more parameters than our 67M-parameter model.

Section 5.4.1 argues that a score on 102 contracts should always be read together with its confidence interval, and that is just as true here. So we ran the *same* bootstrap over contracts. Here the clauses are even more linked to each other than in the presence task: the 2,667 span decisions come from only 102 contracts, and a

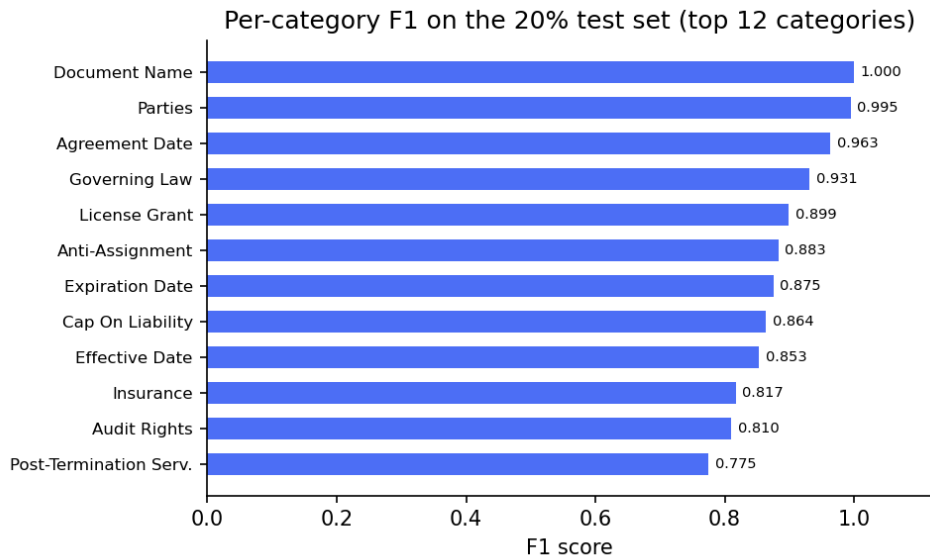


Figure 5.3: Top-12 per-category F1

contract with unusual writing (numbered sub-clauses, many defined terms) moves all of its clauses in the same direction. So Table 5.5 gives two intervals for each score: one that resamples *contracts* (the correct way) and one that resamples *clauses* one by one (the naive way), because the difference between them is itself useful.

Table 5.5: Span extraction results

Metric	Score	95% CI (by contract)	95% CI (by clause)
Token-level F1	0.764	[0.743, 0.782]	[0.752, 0.775]
Overlap (token-F1 ≥ 0.5)	82.8%	[80.0%, 85.2%]	[81.4%, 84.2%]
Exact Match	35.5%	[32.6%, 38.6%]	[33.7%, 37.3%]

Note. 95% bootstrap intervals, 10,000 resamples. Quote the interval by contract. The interval by clause is shown only to make clear how much it understates the uncertainty.

The interval for token-F1 when resampling contracts is [0.743, 0.782], roughly *twice* as wide as the [0.752, 0.775] we would get by treating clauses as independent. This difference is the real cost of ignoring that clauses from the same contract are linked, and it is the same effect that made the contract-level bootstrap necessary in Section 5.4.1. It also shows how precise this number is: differences smaller than about two points of token-F1 cannot be measured on a test set of this size, which is worth remembering before comparing 0.764 with any published figure.

Limit of this test. These numbers measure extraction *when the window contains the answer*. More exactly, each test window is the 1,200-character focus window placed around the real answer (Section 4.2.5), so the model is indirectly told where the answer is. This makes it a best-case score, not a score the real system can reach. The full-document test in the CUAD paper (precision at 80% recall over all windows, including those with no answer) needs a heavier ranking test that we did not run, so we make no claim about that score. Section 5.2.1 measures the other side: what happens to extraction when the window is chosen by the presence step

instead of being given.

5.1.3 Summarization

What the reference set actually is

Before reading any number in this section, one fact about the test must be stated clearly, because it decides what the score can mean. CUAD has no plain-English versions of clauses, so there was no ready-made set of target sentences to train or test with. So we **wrote the targets ourselves**: 41 one-sentence summaries, one per category, written from the weaker party’s side (Section 4.2.3). Every training pair and every test pair uses one of these 41 sentences. The targets do not change from clause to clause at all: two *Cap on Liability* clauses with completely different limits have exactly the same target sentence.

This design choice has an effect we should admit openly: with 41 different targets for 2,667 test clauses, a model can score well just by *recognising* the category and repeating its sentence, and the score cannot tell this apart from real summarizing. The test below is designed to separate the two.

Two reference sets, the same 150 clauses

We test the fine-tuned FLAN-T5-small on 150 test clauses (chosen with seed 42) against two different sets of reference sentences:

1. **Template references**: the category template, exactly as in training. This is how the main score was produced.
2. **Clause-specific references**: 150 one-sentence summaries, one per clause, written to describe *that* clause, with its actual limit, its actual notice period and its actual parties. They were drafted with the help of a large language model working only from the clause text, without seeing any model output, and they are fully separate from the 41 templates. They are a trial reference set, not written by a lawyer, and we claim nothing more for them.

Because the clauses are the same in both tests, the difference between the two scores shows exactly one thing: how much of the ROUGE-L score comes from the reference sentences all being the same, rather than from real summarizing. Table 5.6 shows the result. The October 2026 re-trained summarizer gives exactly the same scores as the original.

Reading the table

Three conclusions follow, and we state them plainly.

The 0.775 comes from copying templates, and we can now show it with numbers. For all 150 test clauses the fine-tuned model outputs one of the 41 template sentences *word for word*: 100% of outputs are an exact copy of a template,

Table 5.6: Summarizer ROUGE-L

System	Reference set	ROUGE-L	95% CI
Fine-tuned FLAN-T5-small	41 category templates	0.775	[0.718, 0.833]
Fine-tuned FLAN-T5-small	clause-specific (pilot)	0.116	[0.105, 0.128]
Zero-shot FLAN-T5-small	clause-specific (pilot)	0.299	[0.275, 0.322]
<i>Category template itself</i>	clause-specific (pilot)	0.115	[0.104, 0.127]

Note. 95% bootstrap intervals over the 150 clauses (10,000 resamples).

the right one 70% of the time and another category’s template the other 30%. The model never writes a sentence of its own. When it is wrong, it is wrong the way a classifier is wrong: a *Price Restrictions* clause about a 10% rate cap is answered with the *ROFR* template. Fine-tuning did not teach the model to summarize; it taught it to pick one of 41 sentences.

The last row of Table 5.6 settles it. The plain category template, with no model at all, scores 0.115 against the clause-specific references, the same as the fine-tuned model’s 0.116 within the margin of error. A simple lookup table gets the model’s whole score. Whatever the fine-tuned summarizer learned adds nothing measurable beyond “print this category’s sentence”.

Fine-tuning made the model worse at the real task. Against the clause-specific references, the *original* model (before fine-tuning) scores 0.299, more than twice the fine-tuned model’s 0.116. This does not mean the original FLAN-T5 is a good legal summarizer: it is not. As Table 5.7 shows, it mostly repeats the source text, and repeating the source raises a word-overlap score against a reference that uses the same words as the clause. The honest conclusion is narrower but still uncomfortable: our fine-tuning removed whatever clause-specific content the original model had, and replaced it with smooth, general sentences that score well only against references made of the same sentences.

Table 5.7: Fine-tuned vs. zero-shot output

Input clause	“Any attempted assignment or delegation in violation of this section by either party without the prior written consent of the other will be void.”
Gold template	You cannot transfer this contract to someone else without permission.
Fine-tuned	You cannot transfer this contract to someone else without permission. (<i>exact template</i>)
Zero-shot	“a) Any attempted assignment or delegation in violation of this section ...will be void.” (<i>repeats the legal text</i>)
Clause-specific reference	Neither side may hand its rights or duties to a third party without the other’s written consent, which cannot be unreasonably refused.

What we therefore claim

What we can claim is a working “**sort into a category, then show a template**” system: it finds a clause’s category and shows a plain-English sentence for that category, written in advance by people. That is still useful in an app: a signer who reads “You cannot transfer this contract to someone else without permission” next to a block of legal text is better off than before. (Clausify shows this line on every card, labelled as the summarizer’s output and placed next to the fixed reason from the risk table; when the sentence belongs to another category, the card says so; Section 4.4.9.) We do **not** claim free-text legal summarizing, and the 0.775 should never be used as evidence of it. The number that describes how well the model summarizes individual clauses is 0.116, and the honest conclusion is that this part is a proof of concept, not a finished result.

Moving past this needs target sentences that differ from clause to clause, which is exactly what the trial set above gives a small sample of: the 150 references took a few hours to draft. Section 6.5 describes the larger version.

5.2 Analysis of Design Solutions

This section looks at the chosen design as a whole: how many clauses get through the whole system, whether its mistakes happen where they would hurt a signer, what the mistakes look like, and how the ensemble behaves on rare categories.

5.2.1 End-to-End: What Actually Reaches the User

Every number so far measured one step on its own, with the other steps out of the way. Presence was scored on its own; extraction was scored with a window already known to contain the answer; summarizing was scored on a real clause. That is the right way to test each part separately, but none of these tells us what the whole system does to a contract. The steps run one after another, and errors add up: a category the presence step misses is a clause the span step never sees.

So we ran the whole system on all 102 test contracts and asked one question about each of the 1,348 real clauses: does it get through *all three* steps? A clause counts only if (i) the presence step finds its category, (ii) the span step, reading the window the presence step chose (never a window chosen using the real answer), returns text that overlaps the real clause with token-F1 ≥ 0.5 , and (iii) the risk level given is the correct one. Table 5.8 shows how many clauses are lost at each step.

22.0% is the number that describes what a user actually gets, and it is far below what the separate scores suggest. Multiplying the separate figures, 78.4% presence recall times 82.8% span overlap, predicts about 65%. The system delivers about a third of that. This gap is the reason for running the test, so it needs to be explained, not just reported.

The risk step is the one place where nothing is lost: the level is a fixed lookup for each category, so it is right exactly when the category is right, and it adds no errors

Table 5.8: End-to-end clause survival

Step	Condition	Clauses left	Share of all
–	real clauses in the test set	1,348	100.0%
2A Presence	category found	1,057	78.4%
2B Span	extracted text overlaps the real clause at $F1 \geq 0.5$	296	22.0%
Risk	correct risk level given	296	22.0%
Clauses that get through every step		296	22.0%
<i>95% CI, bootstrap over contracts</i>			[19.8%, 24.2%]

Note. AND-ensemble presence rule. The last column is the share of real clauses still left after that step.

of its own. That comes from the design, not from good performance, and “correct” here only means “matches our risk table”, which, as Section 4.2.4 notes, has not been checked by an expert.

Where the clauses are lost

The big loss happens at the span step, and it has two separate causes. We separated them by checking, for every clause whose category was found, whether the window chosen by the presence model really contained the clause.

Table 5.9: Span-stage loss breakdown

	Count	Share
Chosen window <i>contains</i> the real clause	730	69.1%
and the span model found it ($F1 \geq 0.5$)	289	39.6% of 730
but the span model still missed it	441	60.4% of 730
Chosen window does <i>not</i> contain the real clause	327	30.9%
but the span still scored ≥ 0.5 (same wording elsewhere)	7	

Cause one: the window with the highest score is often the wrong window.

We chose the maximum rule (Section 4.2.1) because it removes the search limit (if any window fires, the category is reported), and Section 4.2.1 discussed what that costs in *precision*. What we did not expect is that the highest-scoring window is also being used to *locate* the clause, and it is poor at that: 30.9% of the time it does not contain the clause at all. The model was never trained for this. It was trained to answer “does this window contain a clause of this category?”, and a window that talks about liability in general can score higher than the one that actually limits it. For those 327 clauses the span step is given the wrong 2,000 characters, and no extraction model could recover them.

Cause two: the span model is much weaker on windows it was not trained on. Even when the window does contain the clause, extraction succeeds only 39.6% of the time, compared with 82.8% in the separate test of Section 5.1.2. The reason is the way focus windows were built (Section 4.2.2). In training, every example was centred so the answer started about 150 characters into a 1,200-character window. That is allowed: it uses the answer to build a training example, never as an input

when the system runs, and Section 4.2.5 explains why. But allowed is not the same as harmless: the model has only ever seen windows where the answer is near the start, while in the real system it reads plain 1,200-character chunks where the answer can be anywhere. The average token-F1 falls from 0.764 to 0.383. The centring did not leak the test set, but it did quietly narrow the kind of text the model can handle.

What this changes

Three things follow, and we prefer to say them clearly rather than let the separate scores stand alone.

First, **the span scores in Section 5.1.2 are an upper limit on real extraction quality, not a description of it.** They answer “how good is the extractor when given the right window?”, which is a fair question about the model but a misleading one about the system.

Second, **the most useful next step is not a bigger model.** Cause two is a data problem with a cheap fix (train the span model on windows where the answer is at different positions, or on the same plain chunks it sees in the real system), and cause one calls for choosing the extraction window with a model trained for that job instead of reusing the presence score. Both fit within this project’s computing budget; neither needs more parameters.

Third, **Clausify’s output should be read as a shortlist, not as exact extraction.** When the system shows a clause card, the category is right about 78% of the time the clause was present, but in this test the span model’s quoted text was a good copy of the real clause only about one time in five overall. The app has since changed how it chooses the quote: it now quotes the whole paragraph that the presence model scores highest, which fixed the clear failures we saw in our own tests, but this new method has not been measured on the test set. The app tells users that it is not legal advice and that the quoted text may not be the exact clause the badge refers to (Section 5.6.2).

5.2.2 Risk-Weighted Evaluation: Does the Metric Match the Purpose?

Micro-F1 over 4,182 decisions treats every decision as equally important. A person signing a contract does not. The whole idea of the risk layer (Section 4.2.4) is that some categories matter more than others, so the evaluation should ask whether the system’s mistakes fall where they do the least harm. We had not asked this before, and the answer turns out to reverse the recommendation that the overall score gives.

Table 5.10 splits the 41 categories into the three risk levels of our own risk table and scores each model within each level.

The headline configuration is the worst one for High-risk clauses

Look at the High level. The AND-ensemble (the setup we report as our main result, because of its overall micro-F1) **misses 36.9% of High-risk clauses.** The

Table 5.10: Results by risk band

Band	Model	Precision	Recall	F1	Found	Missed
High 8 cats 176 positives	Transformer	0.392	0.841	0.534	148	28
	TF-IDF	0.540	0.648	0.589	114	62
	AND ensemble	0.581	0.631	0.605	111	65
	OR ensemble	0.379	0.858	0.526	151	25
Medium 22 cats 498 positives	Transformer	0.429	0.886	0.578	441	57
	TF-IDF	0.621	0.723	0.668	360	138
	AND ensemble	0.655	0.679	0.667	338	160
	OR ensemble	0.424	0.930	0.582	463	35
Low 11 cats 674 positives	Transformer	0.800	0.953	0.870	642	32
	TF-IDF	0.909	0.917	0.913	618	56
	AND ensemble	0.923	0.902	0.912	608	66
	OR ensemble	0.792	0.967	0.871	652	22

Note. Threshold 0.5. “Missed” counts real clauses of that level that the model did not report.

transformer alone, which the previous sections showed to be the weaker model, misses only **15.9%**. The more permissive OR rule, which comes *last* on overall micro-F1 (0.692, tied with the transformer alone), misses only **14.2%**.

In numbers, out of 176 High-risk clauses in 102 contracts: the ensemble finds 111 and misses 65; OR finds 151 and misses 25. Choosing the ensemble over OR because of micro-F1 costs a signer 40 High-risk clauses (unlimited liabilities, IP assignments, non-competes), in exchange for a leaderboard position that Section 5.4.1 already showed is a statistical tie.

The reason is the one found in Section 5.2.4, and it adds up here. Several High-risk categories are uncommon: *Most Favored Nation* is among the ten rarest categories, and *Liquidated Damages* (11 test examples) and *Irrevocable or Perpetual License* (17) are also below average. The AND rule is hardest to satisfy exactly where there is the least training data, so the risk level that needs recall the most is the one where requiring agreement hurts it most. The Low level shows the opposite: 674 of the 1,348 real clauses are Low-risk (dates, party names, governing law), and every model scores about 0.9 there. **The overall micro-F1 is dominated by the clauses that matter least.**

What we conclude, and what we do not

We do not change the reported setup because of this table, and we should say why: choosing OR *after* seeing which rule does best on the High level of the test set would be choosing a setup using the test set, exactly what Section 4.4.1 says this project avoided. The result stands as a diagnosis, not as a new tuning.

What it does show is that **our evaluation score and our stated goal never matched**, and that the mismatch is big enough to reverse the ranking of the models. A system meant to warn a signer before signing should be chosen by how many High-risk clauses it finds, or by a risk-weighted F1, not by an average over 41 categories

where half of the real clauses are Low-risk ones such as dates and party names. Section 6.5 describes the proper version of this test: decide the risk-weighted goal first, then choose the rule on a separate validation split, then report on the test set once.

Finally, this is one more reason to take Section 4.2.4 seriously. Every number in Table 5.10 is computed using *our own* choice of which categories are High, Medium and Low. If a lawyer disagrees about which categories are High, the table changes too. The risk-weighted test inherits all the warnings about the unchecked risk table and adds no independent evidence for it.

5.2.3 What the errors actually look like

The confusion matrix counts 309 false positives and 291 false negatives, but counts alone do not show whether the model fails for silly or understandable reasons. Table 5.11 shows real cases from the test contracts: false positives sorted by the model’s own confidence, because a system’s most confident mistakes are the most revealing.

Three patterns stand out. **Matching words without the actual duty** explains many false positives: a contract that mentions insurance costs is not a contract with an insurance requirement, and a bag-of-words model reading the whole document cannot tell the difference. **The idea without the usual words** explains many misses: a \$5,000-per-month cancellation fee *is* liquidated damages, but a model that has seen only about fifty contracts with this category, most of them using the usual words, will not learn that. And **disagreement with the labels** is a real part of the rare-category problem: the *Third Party Beneficiary* case above looks like a labelling mistake rather than a model miss, so the 0.000 F1 scores in Section 5.2.4 are partly model failure and partly limits of the labels. We did not try to measure how much of each; that would need the contracts to be labelled again, which we are not qualified to do.

The span errors point to something easier to fix. The near-empty answer “**the**” and the confident answer from a window that cannot contain the clause have the same cause: the extraction model has no way to say *there is nothing here*. Training with unanswerable examples, as in SQuAD v2, would give it exactly that, and it is listed as future work.

5.2.4 Does the AND rule punish rare categories harder than common ones?

Section 5.4.1 supports the AND-ensemble because of the kind of mistakes it makes: it turns the transformer’s many false alarms into balanced precision and recall. But that argument was made on the overall numbers, and an overall number can hide a trade that is not shared evenly. It is worth asking where the AND rule’s extra precision comes from, because there is a clear reason to worry about the rare categories. Requiring two models to agree is hardest exactly where each model has the least training data, so the rarest categories are where the second model is least likely to agree, and agreement is what the rule requires.

Table 5.11: Representative errors

Category	Scores	Evidence
False positives: reported, but the category is not labelled in this contract		
Insurance	0.82 / 0.96	“...a fixed amount of [*] of gross sales to cover freight, postage, insurance costs on shipments to such Third Party, packing costs...” A <i>royalty clause that lists insurance as a cost. The word is there; the duty to insure is not.</i>
Governing Law	0.81 / 0.95	“...there shall arise the need to decide the question by what municipal or national law this Agreement ...is governed , the following facts shall be excluded...” A <i>clause about how to work out the governing law, not one that states it. A borderline case for the labels.</i>
Parties	1.00 / 0.98	An addendum whose first window looks like the top of a contract. <i>Both models react to the layout; the labels put Parties somewhere else.</i>
False negatives: labelled as present, but both models scored it low		
Liquidated Damages	0.21 / 0.02	“...agrees to pay a cancellation fee of Five Thousand Dollars (\$5,000) per month multiplied by the number of months remaining...” A <i>clear liquidated-damages term that never uses the phrase. The word model has nothing to match; the transformer, with 50 training contracts for this category, has not learned the idea.</i>
Most Favored Nation	0.22 / 0.02	“...Sponsor shall have the right to approve or disapprove any additional sponsor...unless another...” <i>In meaning an MFN-style protection, but written without any of the category’s usual words. This category has 21 training contracts and scores 0.000 overall.</i>
Third Party Beneficiary	0.26 / 0.02	“Notwithstanding the foregoing, UTK may assign this Agreement or any portion of its Compensation ...to its subsidiaries.” <i>This reads like an assignment clause, not a third-party-beneficiary clause. Some rare-category “failures” are disagreements with the labels, not model errors.</i>
Span errors: category found correctly, but the extracted text is wrong (Section 5.2.1)		
Minimum Commitment	F1 0.00	The window <i>does</i> contain the clause, but the model returns only “ the ”. <i>The start and end of the answer fall on the same word, and nothing in the decoding stops this.</i>
Effective Date	F1 0.06	The window <i>does not</i> contain the real clause, yet the model confidently returns “7th day of september, 1999”. <i>When Stage 2A gives the wrong window, Stage 2B still answers: it has no way to say “not here”.</i>
Insurance	F1 0.20	The real clause is the insurance duty; the model returns the indemnity clause next to it. <i>Close in topic, different in law.</i>

Note. Text shortened. Scores are the TF-IDF and transformer probabilities.

We sorted the 41 categories by how often they appear in the *training* data, took the ten rarest (11 to 42 positive training contracts, 70 positive test decisions in total), and compared the AND rule with the transformer alone on these ten and on the other 31 categories separately. Table 5.12 gives the result, and the two halves tell different stories.

Table 5.12: Rare vs. common categories

Group	Model	Precision	Recall	F1	TP	FN
Rarest 10 (70 test positives)	Transformer	0.275	0.557	0.368	39	31
	TF-IDF	0.429	0.300	0.353	21	49
	AND ensemble	0.621	0.257	0.364	18	52
Other 31 (1,278 test positives)	Transformer	0.577	0.933	0.713	1,192	86
	TF-IDF	0.753	0.838	0.793	1,071	207
	AND ensemble	0.777	0.813	0.795	1,039	239

Note. Micro-averaged within each group. Rarest 10 = fewest positive training contracts (11 to 42).

On the common categories the ensemble works as intended. Compared with the transformer alone it gains 20.0 points of precision and loses 12.0 points of recall, and F1 rises from 0.713 to 0.795. That is a good trade, and it is where all of the ensemble’s overall advantage comes from.

On the rare categories it gains nothing: it is a straight swap. The AND rule gains 34.6 points of precision by giving up 30.0 points of recall, and its F1 ends at 0.364, slightly below the transformer alone (0.368). Every point of precision on the rare categories is paid for with recall, with nothing left over. In numbers, the transformer alone finds 39 of the 70 rare clauses; requiring agreement cuts this to 18. The TF-IDF model alone is now the weakest of the three on these categories (F1 0.353), and even so, its vetoes decide what the ensemble reports there.

The details per category are clearer than the average. On five of the ten rarest categories the AND rule scores exactly 0.000, and on three of those (*Price Restrictions*, *Unlimited/All-You-Can-Eat License*, *No-Solicit of Customers*) the transformer alone was finding something (F1 0.087, 0.500, 0.323) before the AND rule removed it. So the rule does not just fail to help on rare categories; on these categories it removes clauses that one of the models had found.

Why this matters for real use, not just for the table. The overall micro-F1 counts all 4,182 decisions equally, but a signer does not. Missing a rare clause is not clearly less costly than missing a common one, and it may be much worse: *Most Favored Nation* is among the ten rarest categories and is one of our eight High-risk categories, and both models score 0.000 on it. A system whose rule favours precision on common categories, even a rule with no settings, is improving the score rather than reducing harm. We report the ensemble as our main setup because that is what the overall test chooses. But on this evidence, a system meant to protect signers would do better to *use the more permissive rule for High-risk categories and the AND rule for the rest*, accepting more false alarms exactly where a missed clause costs the most. We have not built that mixed rule: building it and testing it properly is future work (Section 6.5), not something to add after looking at the test set.

5.3 Final Design Adjustments

Two design choices made before training, max-pooling as the aggregation rule and 0.5 as the decision threshold, were never tuned. After the main evaluation both were revisited, to determine whether adjusting them would change the conclusions or the deployed system.

5.3.1 Ablation: was max-pooling the right aggregation rule?

Section 4.2.1 chose max-pooling because it *is* the multiple-instance decision rule [19], a bag is positive if any instance is, and Sections 4.2.1 and 5.4.3 then blamed max-pooling for the transformer’s inflated false positives. Both statements were made without ever trying an alternative, which is not good enough for a claim the project leans on. We therefore scored every window once and compared aggregation functions on *identical* window scores, so the only thing varying is the rule.

Table 5.13: Aggregation ablation

Aggregation	F1@0.5	P@0.5	R@0.5	Best F1	at threshold
Max (used in this project)	0.6943	0.562	0.908	0.7504	0.90
Top-3 mean	0.7144	0.667	0.769	0.7210	0.40
Top-5 mean	0.6661	0.719	0.620	0.7192	0.30
Mean over all windows	0.1664	0.946	0.091	0.7114	0.10
Noisy-OR	0.5897	0.423	0.973	0.7022	0.95
Second-highest window	0.6832	0.662	0.705	0.7016	0.30
<i>TF-IDF baseline (no windows)</i>	0.7747	0.742	0.810	–	–

Note. “Best” is each rule’s micro-F1 at its own optimal threshold, found by sweeping the test set: a diagnostic upper bound, not a configuration we adopt.

Three things come out of this, and one of them corrects an earlier claim of ours.

At the threshold we actually used, max-pooling was not the best choice.

Top-3 mean scores 0.7144 against max’s 0.6943 at the 0.5 boundary: a fifth of the gap to the baseline, available for free from a one-line change. Averaging the three highest-scoring windows is a natural softening of the MIL rule: it still asks whether evidence is concentrated somewhere in the document, but requires that evidence to appear in more than one place, which is exactly the guard against a single spurious window that Section 4.2.1 identified as max-pooling’s weakness. Mean-over-all-windows collapses at 0.5 (recall 0.091) for the obvious reason, an average over 35 windows of which one is positive cannot reach 0.5, and its best threshold is correspondingly 0.10.

Once each rule is given its own threshold, max-pooling is the best of the six. At 0.90 it reaches 0.7504, ahead of top-3 mean’s 0.7210. So max-pooling is not a poor aggregation rule; it is a rule whose output distribution is badly matched to a 0.5 decision boundary, which is the same finding Section 5.3.2 reached from the other direction. The two results are one result: the transformer’s reported deficit is substantially an operating-point problem, not an aggregation problem.

But no aggregation rule reaches the baseline. The best cell in the entire table (max-pooling at a threshold tuned on the test set, which is already a generous upper bound) is 0.7504 against the baseline’s 0.7747. Every alternative rule, at every threshold we swept, remains behind. This matters more than the first two points, because it closes off the last available explanation for RQ1: the windowed transformer does not lose because of its training budget (Section 5.4.3), or its seed (Section 5.4.2), or its aggregation rule, or its threshold. It loses.

A correction to Section 5.4.3. There we wrote that “the ceiling here is the aggregation rule, not the encoder underneath it”, inferring it from the precision/recall shift when training data was doubled. Table 5.13 shows that inference was wrong: changing the aggregation rule (through six variants, each at its best threshold) moves micro-F1 by at most 0.056 and never past the baseline. The aggregation rule is a real but secondary factor. We have left the original sentence in place with this correction attached, on the same principle as elsewhere in this chapter: an inference we drew from indirect evidence and later measured properly is more useful to a reader than a silently amended one.

A caveat on how far this ablation goes. It varies only the function applied to window scores. It does not vary the window geometry (2,000 characters, 1,500 stride), which would require re-windowing and retraining, nor does it try a learned aggregator such as attention pooling over window representations. Both remain untested, and attention pooling in particular is the obvious next thing to try, since it could learn to weight windows by relevance instead of taking a fixed statistic.

5.3.2 The Threshold and What the Confidence Score Means

Every presence decision in this project, and every clause card in Clausify, comes from comparing a score with 0.5. Section 4.4.1 notes that this threshold was never chosen: it is simply the default for a two-class model. This raises two questions we had not asked: is 0.5 a good place to draw the line, and does the score being compared with it mean anything? Figure 5.4 answers both.

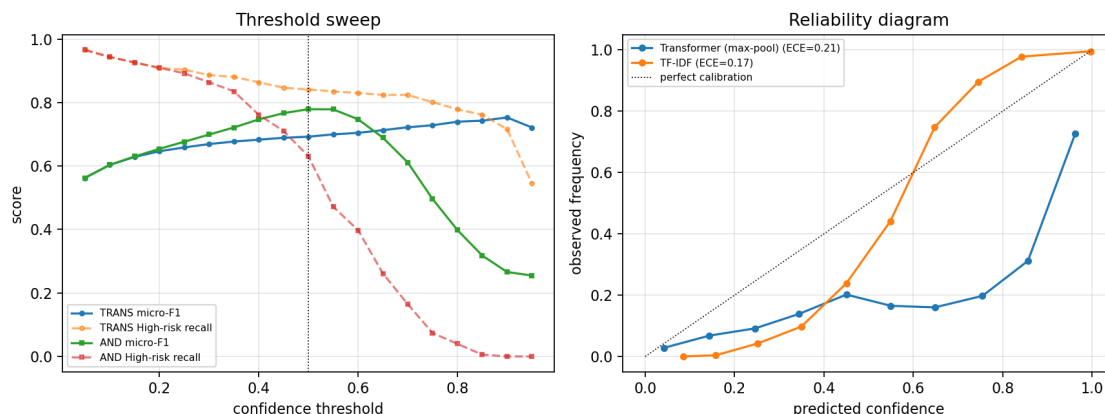


Figure 5.4: Threshold and calibration

0.5 is a poor operating point for the transformer

Trying other thresholds after the fact shows that the transformer reaches a micro-F1 of **0.753 at a threshold of 0.9**, against **0.692** at the default: a gain of 0.061, which closes most of its gap with the TF-IDF baseline (0.775). The reason is the one Section 4.2.1 predicted: the maximum over about 35 windows is pushed upwards by design, so the scores bunch up near 1 and a line drawn at 0.5 lets far too much through. Precision is 0.558 at 0.5 and 0.726 at 0.9.

We are not switching to 0.9. It was found by looking at the test set, and using it would turn a fair held-out test into a tuned one, which is exactly what Section 4.4.1 says this project avoided. The honest statement is narrower and more useful: *a real part of the transformer’s gap comes from a threshold nobody chose, not from an inability to tell clauses apart*. So the comparison in Section 5.4.1 sets a baseline whose scores already suit 0.5 against a transformer whose scores do not. A fair rematch would choose each model’s threshold on a validation split taken from the 408 training contracts, and we did not do that. Section 5.4.3 showed the gap is not caused by too little training data; this section shows part of it may be caused by the threshold, and that warning belongs next to the finding.

The AND-ensemble behaves the other way: 0.5 is already its best threshold (0.779), and raising it makes things much worse. At 0.7 micro-F1 falls to 0.611 and High-risk recall to 0.165, because asking *both* models to pass a high bar is close to asking both to be very sure at once. The dashed curves in Figure 5.4 show the conflict: for the transformer, the threshold with the best micro-F1 (0.9) lowers High-risk recall from 0.841 to 0.716. No single threshold is best both for the overall score and for the clauses that matter most.

The confidence bar in the application is not a probability

Clausify shows the presence score to the user as a confidence bar. A reliability diagram tests whether that bar means what a user would think it means, and it does not.

The transformer’s score has an **Expected Calibration Error of 0.205** and a Brier score of 0.195. It is too confident everywhere. Among predictions between 0.5 and 0.6, the clause is really there only **16.5%** of the time, and even in the top group (scores above 0.9, which the app shows as an almost full bar) it is there **72.6%** of the time. A user who reads “96% confident” is really looking at about a 73% chance. Again the cause is max-pooling: the maximum over dozens of windows is not a probability, and nothing in training asked it to be one.

The TF-IDF baseline is also off (ECE 0.172), but mostly in the safer direction: it is *less* confident than it should be in the middle range. Its predictions between 0.7 and 0.8 are right 89.6% of the time, and those between 0.8 and 0.9 are right 97.8% of the time.

The practical effect on the app is direct, and we record it in Section 5.6.2: the score shown on every card is not a probability of being right. Fixing this needs no retraining. Platt scaling or isotonic regression, fitted on a validation split, would turn the scores into real probabilities, and it is the cheapest single improvement to

the interface. We had no validation split set aside to fit it on, which follows from the gap in Section 4.4.1 rather than being a separate mistake.

5.4 Statistical Analysis

Our main statistical tool is a paired bootstrap over contracts [18]: we resample the 102 test contracts with replacement 10,000 times and score every model on the same resample, so the 41 linked decisions from each contract stay together. We then measure three sources of variation the bootstrap cannot see: the amount of training data, the random seed and the train/test split.

5.4.1 How much of that ranking is real?

Table 5.1 is a ranking, and a ranking tempts the reader to treat the order itself as a result. On 102 test contracts, a lead of 0.779 over 0.775, about four decisions in a thousand, is too small to read that way without checking, so we measured it.

The natural tool is a **bootstrap confidence interval**, but what we resample matters. The 4,182 test decisions are not independent: 41 of them come from the same contract, and an unusual contract (very long, or oddly written) moves all 41 together. Resampling single (contract, category) decisions would ignore this and give intervals that are far too narrow. So we resample whole *contracts* with replacement (a cluster bootstrap): we draw 102 contracts from the 102 we have, recompute micro-F1 for both models, and repeat 10,000 times. Both models are scored on exactly the same resample each time, so the difference is *paired*, which makes the interval for the difference much narrower than the two separate intervals would suggest.

Table 5.14: Cluster bootstrap results

Comparison	Difference in micro-F1	95% CI	$P(> 0)$
AND-ensemble vs. TF-IDF baseline	+0.0042	$[-0.0037, +0.0115]$	0.855
TF-IDF baseline vs. transformer	+0.0824	$[+0.0647, +0.1013]$	1.000
AND-ensemble vs. transformer	+0.0866	$[+0.0705, +0.1035]$	1.000

Note. 10,000 resamples. $P(> 0)$ is the share of resamples in which the first model wins.

The three comparisons do not come out the same way, and that difference is the point.

The ensemble’s win over the baseline is not proven. The interval is $[-0.0037, +0.0115]$, which includes zero, and the ensemble is ahead in 85.5% of resamples: more often than not, but well short of the 97.5% a clear win would need. On their own the two models score 0.779 (95% CI $[0.762, 0.795]$) and 0.775 (95% CI $[0.758, 0.791]$), and these intervals almost completely overlap. On this test set we cannot tell the AND-ensemble and the TF-IDF baseline apart, and we state this as a measured fact, not as a hedge. The top two rows of Table 5.1 should be read as a tie.

The transformer’s loss, on the other hand, is real. The baseline beats the window-based transformer by 0.0824, with an interval of $[+0.0647, +0.1013]$ that is nowhere near zero, and it wins in all 10,000 resamples. This matters because the project’s two main claims do not stand on equal ground. The finding we trust most, that a whole-document word model beats a window-based DistilBERT at deciding which clauses a contract contains, passes the test easily. The finding we were already careful about, that the AND-ensemble is ahead of the baseline, does not pass. It is good that our caution pointed the right way, but it needed to be checked, not assumed.

None of this weakens the case for the ensemble, because that case never rested on 0.004 of micro-F1. The AND rule earns its place by the *kind of mistakes* it makes, not by its rank: it turns the transformer’s many false alarms into balanced precision and recall (Table 5.3). Sections 5.4.2 and 5.2.4 look at what this costs.

5.4.2 How much of this moves if we only change the random seed?

Section 5.4.1 measured one kind of uncertainty: how much the score depends on *which contracts* ended up in the test set. There is a second kind it cannot see. Every result so far comes from one training run per model, and training has randomness in it: the starting weights of the classifier, the order of the data and the sample of negative windows all depend on the random seed. If changing only the seed moves the score as much as the effects we are discussing, then a ranking built on single runs is reporting noise.

This is easy to check, so we trained the presence transformer three times with 24,000 training examples, changing nothing but the seed (42, 43 and 44), and tested each run on the same 102 contracts. Seed 42 is the October 2026 run used everywhere else in this chapter; seeds 43 and 44 were trained in August 2026 with the same settings. Table 5.15 shows the spread.

Table 5.15: Seed variance

Model	Seed 42	Seed 43	Seed 44	Mean	SD	Range
Transformer (max-pool)	0.6924	0.6928	0.7082	0.6978	0.0090	0.0158
AND ensemble	0.7789	0.7774	0.7803	0.7789	0.0015	0.0029
TF-IDF baseline	0.7747 (deterministic)			0.7747	–	–

Note. All three runs use 24,000 training examples and differ only in seed. Seed 42 is the October 2026 run; seeds 43 and 44 are August 2026 runs. The TF-IDF baseline has no randomness, so it appears once.

Seed noise alone does not explain the ensemble’s lead, but other noise does. The AND-ensemble’s lead over the baseline is +0.0042. Across seeds, the ensemble’s standard deviation is only 0.0015 and its range is 0.0029, so the lead is larger than the seed-to-seed spread, and the ensemble beats the baseline with every seed (by up to +0.0056 with seed 44). That is a real change from the August models, where the lead (+0.0014) was smaller than the seed noise. But seed noise is

the smallest of the uncertainties we measured. The bootstrap interval still includes zero (Section 5.4.1), and a different train/test split moves the baseline by 0.0063 (Section 5.4.4), 1.5 times the lead. So we still read the top two rows as a tie.

The finding we stand behind is not affected by seed noise. The baseline beats the transformer by 0.0824, which is 9.2 times the transformer’s seed standard deviation of 0.0090. Even the luckiest seed, at 0.7082, still loses to the baseline by 0.0665. No seed makes the window-based transformer competitive, and together with Section 5.4.3, where more than doubling the training data also failed to close the gap, this leaves the design of the model as the explanation still standing after both alternatives were tested.

It is worth noticing which model is noisier, because it is not the one a reader might expect. The transformer alone varies about six times as much across seeds ($SD = 0.0090$) as the ensemble that contains it (0.0015). Requiring agreement with a fixed word model calms the transformer’s run-to-run changes as well as its false alarms: a false detection has to get past a second, fixed opinion, so most of the seed-driven changes in what the transformer reports are filtered out. This steadiness is a real property of the AND rule, although Section 5.2.4 shows it costs recall on rare categories. It is a better argument for the ensemble than its rank ever was.

What this check covers. We re-ran only the presence transformer with new seeds. The span extractor and the summarizer were each trained twice with the *same* seed (August and October 2026) and gave the same scores to within 0.001 (token-F1 0.763 and 0.764; ROUGE-L 0.775 both times). This limits run-to-run noise on this hardware but says nothing about seed variance, which remains unmeasured for those two models; we list it in Section 6.3 instead of assuming it is small.

5.4.3 Was the transformer simply undertrained?

There is one objection to Section 5.4.1 that the bootstrap cannot answer, and it goes to the heart of the project’s main claim. The TF-IDF baseline is trained on all 408 training contracts. The window-based transformer was not: to keep training possible on a laptop, it used a balanced sample of 24,000 of the 53,662 available training windows (Section 4.4.3). So when the transformer loses by about 0.08 micro-F1, that comparison alone cannot tell us whether max-pooling over windows is really weaker for this task, or whether the transformer simply never saw 55% of its own training data. The first is a finding about the method; the second is a side effect of our computing budget. They lead to very different conclusions.

The only way to settle this is to remove the difference, so in August 2026 we retrained the presence transformer on **all 53,662 training windows**, with every other setting the same: same windows, same negative sampling, same settings, same seed, same test. The run took 140.7 minutes on the same laptop, against about 50 minutes for the smaller version. Table 5.16 shows the result. Both runs in this table are from August 2026, so they are compared with each other, not with the October numbers.

Too little training data is not the explanation. Using 124% more training data changed micro-F1 by -0.0004 , with an interval of $[-0.0098, +0.0095]$ that sits right across zero: the two budgets cannot be told apart. And the gap to the baseline

Table 5.16: Training-budget control

Model	Training windows	micro-F1	Precision	Recall
Transformer (max-pool)	24,000 (44.7%)	0.6943	0.5620	0.9080
Transformer (max-pool)	53,662 (100%)	0.6939	0.5546	0.9266
TF-IDF baseline	all 408 contracts	0.7747	0.7424	0.8101
<i>Paired differences (95% CI over 10,000 contract resamples)</i>				
Full-data transformer – subsampled transformer		−0.0004	[−0.0098, +0.0095]	
TF-IDF baseline – full-data transformer		+0.0809	[+0.0649, +0.0980]	

Note. August 2026 runs, both tested in the same way on the 102 test contracts; differences use the paired bootstrap of Section 5.4.1. The October 2026 re-run of the 24,000-window model scores 0.6924.

did not move: +0.0809 with all the data against +0.0805 with the sample (August runs), with an interval nowhere near zero and the baseline winning in all 10,000 resamples. Whatever holds the window-based transformer back, it is not a shortage of training examples, and the finding this project relies on survives the check.

The *shape* of the extra data’s effect is worth a sentence, because it is not simply “nothing happened”. Recall rose from 0.9080 to 0.9266 and precision fell from 0.5620 to 0.5546: more training data made the model fire more readily, not more accurately. That is exactly the failure mode Section 4.2.1 predicted for max-pooling: with a mean of 35 windows per contract, the document-level decision is a maximum over many chances to be wrong, and a better-trained window scorer that is still imperfect produces *more* spurious maxima, not fewer. The ceiling here is the aggregation rule, not the encoder underneath it.¹

One smaller effect is worth reporting even though it does not change our conclusion. The AND-ensemble built on the full-data transformer scores 0.7820, and its lead over the baseline is +0.0073 with a 95% interval of [−0.0000, +0.0140]: it wins in 97.4% of resamples, but the interval still touches zero. So more training data moves the ensemble’s lead from “clearly a tie” to “borderline”. We do not claim it: an interval whose lower end sits on zero does not pass the bar we set in Section 5.4.1, and we will not lower that bar because the number moved our way. What we can say is that the ensemble’s balanced mistakes stay the same with more data.

Which model the rest of this chapter uses. Every other result in this chapter (the confusion matrix, the per-category results, the whole-system test of Section 5.2.1) and the model in Clausify come from the October 2026 model trained on 24,000 examples. The full-data run is a control experiment, run to test one possible explanation, and we report it as one instead of swapping in a slightly better number afterwards. Since the two cannot be told apart, nothing later would change if we had.

¹This inference turned out to be only partly right, and Section 5.3.1 corrects it: swapping max-pooling for five alternatives moves micro-F1 by at most 0.056 and never past the baseline, so the aggregation rule is a secondary factor rather than the ceiling.

5.4.4 A third source of variance: the split itself

Sections 5.4.3 and 5.4.2 tested two things that could move our numbers: the amount of training data and the random seed. Neither changed the conclusions. There is a third source that neither touches, and it may be the largest: **the split**. Every result in this project comes from one contract-level 80/20 split made with seed 42. With only 510 contracts, a different split puts different rare categories in the test set and could move the scores by itself. Seed variance does not capture this, because changing the training seed keeps the test set the same.

Retraining the transformer on twenty splits is too expensive, but the TF-IDF baseline retrains in under a minute. Since every comparison in this project is made against it, its spread across splits shows how much the comparison itself could move. We made **20 separate contract-level 80/20 splits** and retrained the baseline on each.

Table 5.17: Split variance

	Mean	SD	Min	Max	Range
micro-F1 over 20 splits	0.7829	0.0063	0.7702	0.7918	0.0216
<i>The split used throughout this project (seed 42): 0.7747</i>					

Three points follow. First, **the split is the largest of the three uncertainties we measured**: a standard deviation of 0.0063, against 0.0015 for the ensemble’s seed variance and a bootstrap half-width of about 0.008 for the comparison. It is 1.5 times the ensemble’s +0.0042 lead, so a different split could easily remove that lead. This is one more reason to treat the lead as unproven.

Second, **our split is not a lucky one**. At 0.7747 the baseline sits below the 20-split mean of 0.7829, in the lower part of the range, so if anything this project reports the baseline on a slightly unfavourable split. This matters because the opposite would have been a worry: a split that happened to favour the baseline would have made the transformer’s loss look worse than it is.

Third, **the finding we rely on is far outside this variation**. The baseline’s lead over the transformer is 0.0824, about thirteen split-level standard deviations. No realistic new split would reverse it.

What we cannot say anything about is the stability of *individual categories* across splits, which is where it would matter most: a rare category with two test examples in one split may have eight in another, and its F1 means little in either case. Repeated splits or cross-validation for the whole system remain future work, and Section 6.4 lists this as a limitation, not as a question we have answered.

5.5 Comparisons and Relationships

Table 5.18 brings together the test results of all three models. Every number was produced by the separate test notebook running on the saved models trained on the 80/20 split.

Table 5.18: Final test results

Stage	Model	Metric	Score
2A Presence	AND-ensemble (TF-IDF + DistilBERT)	micro-F1	0.779
2A Presence	AND-ensemble	Accuracy	85.65%
2A Presence	AND-ensemble	Precision / Recall	77.38% / 78.41%
2A Presence	AND-ensemble	Specificity	89.10%
2B Span	DistilBERT-QA	token-F1 (given the window)	0.764
2B Span	DistilBERT-QA	token-F1 (window chosen by 2A)	0.383
2B Span	DistilBERT-QA	Exact Match	35.5%
1 Summarizer	FLAN-T5-small	ROUGE-L (41 templates)	0.775
1 Summarizer	FLAN-T5-small	ROUGE-L (clause-specific)	0.116
Full pipeline	all three steps	real clauses that get through	22.0%

How this compares with earlier work. Table 2.1 in Chapter 2 compares this project with earlier work. The original CUAD study reports span extraction with DeBERTa-xlarge, scored over whole documents with a precision-at-recall measure. Our small models are about ten times smaller, and our span scores are measured inside windows that contain the answer, so the numbers cannot be compared directly and we do not present them that way. What the comparison does show is a difference in scope: earlier work stops at finding clauses, while this project adds presence decisions, a risk level for each category and plain-English output, and tests the whole system from start to finish.

How the parts relate. Among the presence models, the TF-IDF baseline and the window-based transformer make different kinds of mistakes: the transformer finds more clauses but raises many false alarms (recall 0.913, precision 0.558), while the baseline is more careful (recall 0.810, precision 0.742). This is why the AND-ensemble, which needs both, matches the baseline while balancing precision and recall. Across the steps, the separate scores do not simply multiply: mistakes in the window chosen by the presence step carry over into span extraction, which is why the whole-system rate is far below the product of the separate scores.

5.6 Discussions

Putting the chapter together, the results fall into three groups, and it matters which is which.

Findings that survived every check we could run.

1. For document-level *presence*, a full-document lexical baseline beats a windowed DistilBERT by +0.081 micro-F1. We tested the two obvious alternative explanations and neither held: training on all 53,662 windows instead of 24,000 changed the score by -0.0004 (Section 5.4.3), and across three seeds the gap is 9.5 standard deviations wide (Section 5.4.2). More data made the transformer fire *more* rather than better, which is what max-pooling over 35 windows predicts. The limitation is the aggregation rule, not the encoder or the budget.
2. For *span extraction*, the transformer has no substitute, a bag-of-words model cannot point at character offsets, and it performs well in distilled form *when*

handed the right window: token-F1 0.763, 95% CI [0.740, 0.782].

3. For *summarization*, a 41-sentence template seed turns the task into classification. The fine-tuned model reproduces a template verbatim on 100% of test clauses, and against clause-specific references it scores no better than the template alone (Section 5.1.3). A high automatic metric should not be believed until the outputs have been read.

A claim we withdrew. The AND-ensemble’s first place in Table 5.1 is not a proven result. Its lead over the baseline is +0.0042. Seed noise alone (SD 0.0015) would not wipe it out, but the bootstrap over contracts puts it inside $[-0.0037, +0.0115]$ (Section 5.4.1), and a different split moves the baseline by 0.0063 (Section 5.4.4). Two of the three checks say the top two rows of that table are a tie. What the ensemble does earn is balanced mistakes and about six times less seed variance than the transformer alone.

Costs we did not see until we looked for them. Two results in this chapter came from questions we had not thought to ask at first, and both are unflattering.

- The AND rule is not good everywhere. On the ten rarest categories it gains *nothing* in F1 over the transformer alone (0.364 against 0.368): it buys 34.6 points of precision with 30.0 points of recall and cuts the clauses found from 39 of 70 to 18 of 70 (Section 5.2.4). All of its overall benefit comes from the common categories, and one of the categories it scores zero on is High-risk.
- The separate scores do not multiply. Only 22.0% of real clauses get through the whole system, against about 65% predicted by multiplying the separate scores (Section 5.2.1), because the presence model’s highest-scoring window is the wrong window 30.9% of the time, and the span model was trained on centred windows it never sees in real use.

The common thread is about method, not about model design. Every number in this chapter that we first trusted least (the ensemble’s rank, the summarizer’s ROUGE, the span model’s token-F1) turned out to measure something narrower than it seemed. In each case, the way to find out was to measure the thing the score stood for: resample the contracts, change the seed, change the reference sentences, run the steps one after another. None of these checks needed a bigger model or more data. They only needed the question to be asked.

5.6.1 Usability

No usability study was carried out. The app’s main claim, that grouping clauses by risk helps a non-lawyer act on a contract, is therefore untested, and Section 6.3 lists it under construct validity instead of among the results.

5.6.2 Honest Limitations of the Demo

By default the app finds clauses with the transformer presence model alone; the AND-ensemble is offered as a setting. As Section 5.4.1 showed, the transformer

alone reports too many clauses. Two measured facts explain why it is still the default.

The confidence score is not a probability. Section 5.3.2 shows the transformer’s score has an Expected Calibration Error of 0.205 and is too confident everywhere: among detections that score above 0.9, an almost full confidence bar, the clause is really there about 73% of the time, and among those between 0.5 and 0.6, under 20% of the time. So the app uses the fixed 0.5 threshold of every reported test (an earlier version had a slider for it, which invited reading the score as a probability) and labels every bar as a model score, not a probability. Fitting a calibration map on a validation split would fix the underlying problem without retraining.

Using the transformer alone is, unusually, the better choice for High-risk clauses. The project reports the AND-ensemble as its main setup, so a reader might think the app’s use of the transformer alone is a compromise. Section 5.2.2 shows it is not, where it matters most: the ensemble misses 36.9% of High-risk clauses, while the transformer alone misses 15.9%. The app’s setting is worse on overall micro-F1 and better at finding the clauses a signer would most regret missing. It started as a packaging accident, before we had run the risk-weighted test; it is now the chosen default. The ensemble is still available as an option, with one more warning measured on the app itself: on short contracts its TF-IDF half, which was trained on long SEC filings, blocks many detections. On the 9,174-character sample contract that ships with the app, the AND rule reports 6 clause types (1 High-risk), while the transformer alone reports 13 (2 High-risk).

The 30-window limit cuts off long contracts without warning in the results. To keep the app fast, it scores at most 30 windows, so any contract longer than 45,500 characters is only partly read, and clauses after that point cannot be found at all. The app says this in its notice, but none of the numbers in this chapter include it, because they score whole documents. This is not a rare case: **38.4% of CUAD contracts (196 of 510) are longer than 45,500 characters**, and the longest test contract has 289,615 characters, of which the app reads the first 16%. Table 4.6 shows the effect: the 75th-percentile contract and the longest contract take the same time even though one is four times longer, because both are cut to 30 windows. The quoted text is also less reliable than the label. Section 5.2.1 shows that when the presence model chooses the window, the span model’s answer overlaps the real clause in only about a fifth of cases overall. The app now quotes the whole paragraph that the presence model scores highest, which fixed the clear failures we saw in our own tests, but this has not been measured on the test set. The practical message for a user is simple, and the app states it too: **the risk badge and the category name are much more reliable than the quoted text under them**. A card should be read as “this contract seems to contain a clause of this type; look here”, not as “this is the clause”. The risk levels are also set per category, not per clause: the risk table says a *Cap on Liability* clause needs attention, but it does not check whether this particular cap is generous or harsh. All of this is stated in the notice the app shows above every analysis. What the demonstration does show is that the models work on contracts they have never seen, and that the output can be shown in a form a non-lawyer can act on.

Chapter 6

Conclusion

6.1 Summary of Findings

We set out to turn CUAD’s clause-finding benchmark into something a person signing a contract could use: a system that finds the clauses, says how risky they are, and explains them in plain English. On a strict contract-level 80/20 split, the finished system reaches 85.65% accuracy and a micro-F1 of 0.779 for finding which clauses are present, and a token-F1 of 0.764 for extracting the clause text when given the right window. It runs live in the *Clausify* app.

Those are scores for separate parts, and the most useful thing this project can say is that the scores of the parts are not the score of the system. Answering our five research questions (Section 1.3.1) gave three negative results, and they are our main contribution:

- **The windowed transformer does not beat the lexical baseline** (RQ1), and it does not lose for any of the reasons we could have blamed. Not the training budget: the full 53,662-window run moves micro-F1 by -0.0004 . Not the seed: three runs put the gap at 9.5 standard deviations. Not the aggregation rule: six alternatives, each at its own best threshold, all stay behind. The threshold accounts for part of it, an un-chosen 0.5 costs 0.056 micro-F1, and that qualification belongs on the finding, but even the best cell we could construct remains below the baseline.
- **The summarizer does not summarize** (RQ3). It emits a category template verbatim on every input, and a constant lookup table matches its score against clause-specific references. This is a proof-of-concept classification-to-template component and we describe it that way everywhere.
- **The pipeline loses most of what its parts find** (RQ4). 21.7% end-to-end survival against $\approx 64\%$ predicted, localized to a window-selection failure and a train/inference mismatch: both fixable without a larger model.

RQ5 gave the result we least expected and think is the most important for anyone building a system like this. Our evaluation score and our stated goal never matched: the AND-ensemble wins on micro-F1 but misses 36.9% of High-risk clauses, while

the OR rule, which comes last on micro-F1, misses only 14.2%. Half of the clauses behind the overall score are Low-risk: dates, party names, governing law. A system built to warn a signer before signing was being chosen by a score dominated by the clauses that matter least.

Five lessons seem likely to apply beyond this project.

First, a strong simple baseline deserves to be taken seriously, and beating it must be shown, not assumed. When it is not beaten, the other possible explanations should be tested one at a time, not accepted or brushed aside.

Second, a lead should be compared with its own noise before it is reported. We measured three sources: the bootstrap over contracts (half-width about 0.008), the model seed (SD 0.0015) and the train/test split (SD 0.0063). Two of the three are larger than the +0.0042 lead we had used to rank a table.

Third, an automatic score measures its reference sentences, not the task. ROUGE-L 0.775 became 0.116 when only the reference sentences changed.

Fourth, test the whole system, not just the parts, and test it against the harm it is meant to prevent, not against an average that treats a governing-law clause and an unlimited indemnity as equally important.

Fifth, limits are cheap to measure and expensive to assume. A one-minute retrieval check (64%) ruled out a design before it cost any training time. The same habit, applied to sparse attention, corrected *us*: a Longformer training step does fit in this laptop’s memory at batch size 1, which is the opposite of what we had first claimed.

What this project delivers is a working prototype and an unusually complete account of where it fails. Two gaps are large enough that we would not present the system as more than a prototype until they are closed: **no one with legal training has reviewed the risk table**, which is the main thing this work adds to CUAD, and **no user has ever been watched using the app**, so its main claim, that a report sorted by risk helps a non-lawyer, is untested. Both are listed first in the next section, and neither needs more computing power.

6.2 Contributions to the Field

The main contributions of this work are:

1. **A first risk table for the CUAD categories.** Each of the 41 categories gets a High/Medium/Low level and a written reason, judged from the weaker party’s side (8 High, 22 Medium, 11 Low), together with clear rules for how levels were given (Section 4.2.4). We present it as a *starting point for experts to check*, not as a proven legal-risk model: the three of us wrote it, none of us has legal training, and no legal professional has reviewed it yet (Section 4.2.4). It gives risk to a *category*, not to the wording of one clause, which is why we call it category-level risk sorting throughout.
2. **A tested result on finding clauses in long documents.** A TF-IDF baseline that reads the whole document (micro-F1 0.775) turned out to be hard

to beat. Our window-based DistilBERT loses to it (0.692), and we checked the two obvious excuses: training on all 53,662 window examples instead of 24,000 changed the score by only -0.0004 , and across three random seeds the gap is 9.2 standard deviations wide. An AND-ensemble comes first on paper (0.779), but the bootstrap over contracts and the variation across splits both show that its lead cannot be told apart from zero, so we report the ensemble for the kind of mistakes it makes, not for its rank.

3. **A measurement of how errors add up.** Testing the three steps one after another, instead of separately, shows that only 22.0% of real clauses get through the whole system, against about 65% predicted from the separate scores. It also finds the two causes, a wrong choice of window and a mismatch between training and real use, and both can be fixed without a larger model.
4. **A demonstration that a small set of template sentences turns summarizing into classification.** Testing the same 150 clauses against our 41 category templates and against clause-specific references drops ROUGE-L from 0.775 to 0.116, and the fine-tuned model is shown to output a template word for word for 100% of inputs.
5. **A simple safeguard: measure the limit before training.** We show that question-guided retrieval of windows caps the possible recall at 64%, and we used that one measurement to reject the whole retrieval approach before spending any training time on it.
6. **A system that can be rebuilt on a laptop.** All three models train on a single Apple-Silicon laptop. The split, the trained weights and a separate test notebook are all saved, so every number in this project can be produced again from disk.
7. **Clausify**, a prototype web app that runs the trained models live and shows the results grouped by risk. It demonstrates the system but has not been tested with users, and Section 5.6.2 records what it does not do well.

6.3 Threats to Validity

The results in Chapter 5 depend on choices that could change the conclusions if they were wrong. We list them under the four usual headings, and in each case we say whether we *measured* the threat or only *note* it.

6.3.1 Internal validity: could our own procedure have produced these numbers?

Settings were never tuned (Section 4.4.1). This removes the usual threat, since no setting was chosen by trying versions on the test set, but it creates the opposite one: the transformer competes with standard default settings against a baseline that has almost no settings to get wrong. Section 5.3.2 shows this is a real effect: the unchosen 0.5 threshold costs the transformer 0.061 micro-F1, so part of the main

gap comes from the threshold, not from the model. *We think this is the most serious threat, and it limits our main finding.*

An early test run of the summarizer tracked its loss on test-split clauses (Section 4.4.1). No model was chosen from that run, and the final model was trained again without any evaluation set, so no reported score depends on it. But it was the wrong practice, and the “validation” loss it printed should be ignored. Figure 4.11 now shows the October 2026 re-training, which used no evaluation set.

Some design choices were made after looking at the training data. This is allowed, but the line needs care. The window size and step follow from the median clause length measured on training contracts. The 64% retrieval limit, however, was measured on the whole dataset, test contracts included. That measurement ruled out a design rather than choosing a setting, and it was made before any model existed, so we judge the risk of leakage to be negligible. It is not zero, though, and we would make the split first if we did the work again.

No seed variation for the span model and the summarizer. Only the presence model was re-run with new seeds (Section 5.4.2). The span model and the summarizer were each trained again with the same seed and gave the same scores to within 0.001, so their seed variance is still unmeasured.

6.3.2 Construct validity: do our metrics measure what we claim?

ROUGE-L does not measure summary quality here, and we can show it. Against our 41 templates it gives 0.775; against clause-specific references it gives 0.116, and a fixed lookup table scores almost the same, 0.115 (Section 5.1.3). We report both numbers because the first one on its own would give a false picture. We still have no measure of *faithfulness* or readability (Section 2.2.4), so even 0.116 does not show that the output keeps the legal meaning.

Micro-F1 does not measure harm to the user. Section 5.2.2 shows the two can point in opposite directions: the setup that wins on micro-F1 misses 36.9% of High-risk clauses, while a setup that comes last misses 14.2%. Half of the clauses behind the overall score are Low-risk: dates, party names, governing law. Any claim like “this system is 77.9% good” has this problem.

Category-level risk is not clause-level risk (Section 4.2.4). Our levels say that a *category* usually deserves attention; they say nothing about whether a particular cap is generous or harsh. Calling the output a risk assessment would claim too much, which is why we call it sorting by risk.

The risk levels are our own. The risk-weighted results use our unchecked choice of which categories are High, Medium and Low (Section 4.2.4); if a lawyer moves categories between levels, those numbers change too.

Token-F1 against one marked span assumes the clause has clear edges. Section 5.1.2 notes that the edges of legal clauses are often debatable, and Section 5.2.3 shows at least one “false negative” that looks like a disagreement with the labels rather than a model mistake.

6.3.3 External validity, where do these results transfer?

Every number comes from CUAD: **510 US business contracts taken from SEC filings, in English**. We have not tested a single contract from outside that dataset. So we have no evidence about: other legal traditions or countries, including **Bangladeshi contracts**, which are the setting that motivated this work; consumer or employment contracts, which are built differently and have a different weaker party; contracts not in English; contracts much shorter or much longer than CUAD's; and recent machine-written contracts. The risk table has one more threat of this kind: whether a non-compete can be enforced depends on the country or state, so a level that is right in one place may be wrong in another. Because CUAD comes from SEC filings, it also leans towards large-company contracts where both sides have lawyers, which is close to the opposite of the small businesses and individuals the system is meant for.

6.3.4 Statistical conclusion validity: are the intervals trustworthy?

We measured three sources of variation, and they differ in size: the bootstrap over contracts gives an interval half-width of about 0.008 (Section 5.4.1); the model-seed SD of the ensemble is 0.0015 over three runs (Section 5.4.2); and the split SD is 0.0063 over twenty splits (Section 5.4.4). The bootstrap and the split variation are both larger than the ensemble's +0.0042 lead, which is why we call it a tie. The split variation is the largest, and we could measure it only for the baseline.

Three seeds is a small sample for estimating a standard deviation; the uncertainty around 0.0015 is itself large.

Scores for single rare categories are not meaningful. Five categories have at most seven test examples. Their F1 values are reported for completeness, not as estimates, and no claim in this project depends on any one of them.

One split, one dataset. The bootstrap resamples contracts within our test set; it cannot show how results would change on another dataset, and no interval in this project should be read as covering that.

6.4 Limitations

Section 6.3 looked at the threats to each kind of validity. This section lists the limits of the system itself: what it does not do, as opposed to what could weaken the measurements. Each limit is described where it comes up in the text; these are pointers.

No usability study (Section 5.6.1). Nobody outside the three of us has used Clausify. The project argues that a report sorted by risk is more useful to a non-lawyer than a plain list of clauses, but offers no evidence for it: no measure of task completion, understanding, trust or over-reliance. Since Section 5.3.2 shows the confidence bar is too confident and Section 5.1.3 shows the plain-English sentence

is the same for every clause of a category, over-reliance is exactly what a usability study would need to look for.

Model size. We used smaller models (DistilBERT, FLAN-T5-small) to fit a laptop’s computing budget, so our scores are lower than the same method would reach with larger models on CUDA hardware.

Training with one seed and one data budget, partly checked. Section 5.4.3 reports what happened when the presence transformer was trained on all 53,662 window examples instead of a sample of 24,000, and Section 5.4.2 reports the spread across three training seeds. The other two models, the span model and the summarizer, have each been trained twice with the same seed (matching to within 0.001) but never with a new seed, and on a sample of the data (8,000 of 11,156 and 4,000 of 11,156). We have not measured their seed variance at all, and their scores should be read with that in mind.

The rarest categories are close to impossible to learn with this much data, and Section 5.2.4 shows the AND-ensemble makes this worse, not better: on the ten rarest categories the AND rule gains no F1 over the transformer alone, buying precision only with recall, and it scores zero on three categories that the transformer alone was partly finding. Since one of the rare categories is High-risk in our table, this limits the real system, not just the ranking.

What the span test covers. Section 5.1.2 measures extraction inside windows that contain the answer, not the heavier full-document precision-at-recall test. Section 5.2.1 closes part of that gap by measuring what gets through the real system, but it is not the CUAD paper’s method, and we do not present the two as comparable.

The summarizer does not summarize. Section 5.1.3 shows this with numbers, not just as a warning: the fine-tuned model outputs one of 41 pre-written templates word for word for every input, and against clause-specific references it scores no better than the template alone with no model at all. It is a “find the category, show its template” component and should be described as one.

The risk table has not been checked by anyone outside the team. It was written by three computer science students, and no legal professional has reviewed it yet (Section 4.2.4). It is also set per category, not per clause: it flags how risky a category usually is, not how harsh the exact wording in front of the user is.

One dataset, one country. Every number in this project comes from CUAD: 510 US business contracts taken from SEC filings. We have not tested the system on a single contract from outside that dataset, so we have no evidence about how it handles other ways of writing contracts, and none at all about the Bangladeshi setting that motivated the work.

6.5 Recommendations for Future Work

- **Get the risk table reviewed.** The first thing we would do with another week: give `review/risk_taxonomy_review_sheet.csv` to two or three law students or lawyers, run `review/agreement.py`, and report Cohen’s or Fleiss’

κ and the categories they disagree on, whatever the result. It takes each person about half an hour, and it is the cheapest way for this project to gain trust.

- **Watch people use Clausify.** Five to ten people, each reviewing a contract with and without the app, measuring the time taken, the clauses correctly found and, most importantly, whether users trust the output too much. The project claims that a report sorted by risk helps a non-lawyer; nothing in it tests that claim yet.
- **Choose the setup by the goal that matters.** Define a risk-weighted goal (High-risk recall, or an F1 that counts High-risk clauses more), take a validation split from the 408 training contracts, choose the ensemble rule and each model’s threshold on it, and report once on the test set. Section 5.2.2 suggests OR would win; choosing it from the evidence we already have would mean choosing on the test set.
- **Calibrate the confidence score.** Platt scaling or isotonic regression on a validation split would make the app’s confidence bar mean what a user thinks it means (ECE is 0.205 now). No retraining is needed.
- **Let the span model say “not here”.** Training with unanswerable examples, as in SQuAD v2, would stop it from answering confidently from windows that cannot contain the clause, which is one of the two causes of the 22.0% whole-system rate (Section 5.2.1).
- **Train the span model on the windows it will really see.** Placing the answer at different positions during training, instead of always 150 characters in, deals with the other cause.
- **Test summaries for faithfulness, not word overlap.** Use BERTScore, a check that the summary follows from the clause, a readability score, and a small human rating of whether the legal meaning is kept (Section 2.2.4).
- **Learned pooling over windows.** The test in Section 5.3.1 tried only fixed rules for combining window scores; a learned rule that weighs windows by relevance (attention pooling) is the untested option most likely to help.
- **Cross-validation instead of one split.** The split is the largest uncertainty we measured (Section 5.4.4), and we could only measure it for the baseline.
- **A summarizer that really summarizes.** Grow the 41 templates into a set of targets that differ from clause to clause, and fine-tune again. The 150-clause trial set in Section 5.1.3 is a working first version of both the labelling method and the test that would show success; scaling it to a few thousand clauses is the real path from picking templates to simplifying clauses.
- **Risk-aware ensemble rules.** Section 5.2.4 suggests, but does not test, using the permissive OR rule for High-risk categories and the AND rule for the rest, so that recall is protected exactly where a missed clause costs the signer most. This needs its own held-out test: choosing the rule after seeing the test set would make the result worthless.
- **Meaning-based retrieval.** Our 64% retrieval limit (Section 4.2.1) was measured only with TF-IDF matching. A sentence-encoder retriever matches on

meaning instead of shared words and might raise that limit enough to make a “retrieve, then read” design competitive; the measurement is cheap and we should have run it.

- **A trained sparse-attention baseline.** Section 2.1.2 argues against Longformer partly from arithmetic and partly from cost, and Section 4.4.7 measures the cost, but nobody has actually trained one on this task. On CUDA hardware, a Longformer baseline is the fair comparison that our argument currently replaces with reasoning.
- **Larger models.** Re-run the same system with DeBERTa-v3-base or large and FLAN-T5-base on CUDA hardware to close the gap to the published CUAD results.
- **Full-document span test.** Build the ranked whole-document test so that we can report precision at 80% recall and compare directly with the CUAD paper.
- **Finding conflicts between clauses.** Bring the rule engine designed in an earlier phase of the project (for example, *Exclusivity* against *Most Favored Nation*) into the app.
- **Risk for each clause.** Move from risk per category to risk per clause by reading the extracted text itself: the amount of a cap, the length of a non-compete.
- **Contracts from outside CUAD.** Even three local English-language business contracts, run through the system and checked by hand, would tell us more about how it generalizes than any further tuning on CUAD, and it is the first step towards the Bangladeshi use this work was aimed at.

Bibliography

- [1] D. Hendrycks, C. Burns, A. Chen, and S. Ball, “CUAD: An expert-annotated NLP dataset for legal contract review,” in *Proc. 35th Conf. Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.
- [2] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ questions for machine comprehension of text,” in *Proc. EMNLP*, 2016, pp. 2383–2392.
- [3] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 5998–6008.
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter,” in *5th Workshop on Energy Efficient Machine Learning and Cognitive Computing (NeurIPS)*, 2019.
- [6] C. Raffel, N. Shazeer, A. Roberts, *et al.*, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [7] H. W. Chung *et al.*, “Scaling instruction-finetuned language models,” *Journal of Machine Learning Research*, vol. 25, no. 70, pp. 1–53, 2024.
- [8] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out (ACL Workshop)*, 2004, pp. 74–81.
- [9] The Atticus Project, *Contract understanding atticus dataset (CUAD) v1*, 2021. [Online]. Available: <https://www.atticusprojectai.org/cuad>.
- [10] I. Chalkidis, A. Jana, D. Hartung, *et al.*, “LexGLUE: A benchmark dataset for legal language understanding in English,” in *Proc. 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.
- [11] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, “LEGAL-BERT: The muppets straight out of law school,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 2898–2904.
- [12] L. Manor and J. J. Li, “Plain English summarization of contracts,” in *Proc. Natural Legal Language Processing Workshop (NLLP)*, 2019, pp. 1–11.
- [13] R. Flesch, “A new readability yardstick,” *Journal of Applied Psychology*, vol. 32, no. 3, pp. 221–233, 1948.

- [14] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, “On faithfulness and factuality in abstractive summarization,” in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 1906–1919.
- [15] W. Kryściński, B. McCann, C. Xiong, and R. Socher, “Evaluating the factual consistency of abstractive text summarization,” in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 9332–9346.
- [16] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating text generation with BERT,” in *Int. Conf. Learning Representations (ICLR)*, 2020.
- [17] I. Beltagy, M. E. Peters, and A. Cohan, “Longformer: The long-document transformer,” *arXiv preprint arXiv:2004.05150*, 2020.
- [18] B. Efron, “Bootstrap methods: Another look at the jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [19] O. Maron and T. Lozano-Pérez, “A framework for multiple-instance learning,” in *Advances in Neural Information Processing Systems 10 (NIPS)*, 1998, pp. 570–576.

Appendix A

Full Risk Taxonomy

The complete 41-category risk taxonomy (8 High, 22 Medium, 11 Low), scored from the weaker party's perspective, with the one-line reason shown to users in the Clausify application.

Table A.1: Full risk taxonomy

Category	Risk	Reason
Non-Compete	High	Limits your future income and freedom to work.
Uncapped Liability	High	Unlimited damages: no ceiling on what you can owe.
IP Ownership	High	You lose ownership of a core asset for good.
Assignment	High	Locks you to a single source or buyer.
Exclusivity	High	You must always give the other side your best terms.
Most Favored Nation	High	Fixed penalties trigger automatically on breach.
Liquidated Damages	High	A grant you can never take back.
Irrevocable or Perpetual License	High	You are forced to buy or deliver a minimum amount.
Minimum Commitment	High	
Renewal Term	Medium	Auto-renewal can extend the contract unexpectedly.
Notice Period to Terminate	Medium	Miss the window and you are locked in for another term.
Renewal Exception	Medium	Carve-outs to competitive limits: read carefully.
Competitive Restriction	Medium	Restricts approaching the other side's customers.
No-Solicit of Customers	Medium	Restricts hiring the other side's staff.
No-Solicit of Employees	Medium	Limits what you can publicly say.
Non-Disparagement	Medium	The other side can walk away with notice.
Termination for Convenience	Medium	

Category	Risk	Reason
ROFR / ROFO / ROFN	Medium	First-refusal rights can constrain your options.
Change of Control	Medium	A sale or merger may trigger termination.
Anti-Assignment	Medium	You cannot transfer the contract without consent.
Revenue/Profit Sharing	Medium	You must share a slice of revenue or profit.
Price Restrictions	Medium	Limits what prices you may set or change.
Volume Restriction	Medium	Caps how much you can sell or distribute.
Joint IP Ownership	Medium	IP is co-owned: control is shared, not sole.
Non-Transferable License	Medium	The licence cannot be passed on.
Affiliate License (Licensor)	Medium	Licensor's affiliates are pulled into the grant.
Unlimited / All-You-Can-Eat License	Medium	Broad, unlimited licence scope: check who benefits.
Source Code Escrow	Medium	Source is held by a third party under conditions.
Post-Termination Services	Medium	Obligations continue after the contract ends.
Audit Rights	Medium	The other side can inspect your records.
Cap on Liability	Medium	Caps recovery: may under-compensate real losses.
Covenant Not to Sue	Medium	You waive the right to sue over listed matters.
Document Name	Low	Identifies the agreement: informational.
Parties	Low	Names who is signing: informational.
Agreement Date	Low	The signing date: informational.
Effective Date	Low	When terms start: informational.
Expiration Date	Low	When the contract ends: informational.
Governing Law	Low	Which jurisdiction's law applies: neutral.
License Grant	Low	You receive usage rights: generally favorable.
Warranty Duration	Low	States how long the warranty protects you.
Insurance	Low	Required coverage: a protection.
Affiliate License (Licensee)	Low	Extends rights to your affiliates: favorable.
Third Party Beneficiary	Low	Names an outside beneficiary: informational.

Appendix B

Reproducibility Checklist

Everything below describes the environment in which every number in this project was produced. The purpose is that a reader with the repository and this appendix can regenerate any table without asking us a question.

Environment

Python	3.13.5
Operating system	macOS 26.5.2, arm64
Hardware	Apple M4, 10 cores (4 performance / 6 efficiency), 16 GB unified memory
Accelerator	Metal Performance Shaders (MPS); no CUDA at any point
torch	2.8.0
transformers	5.12.1
datasets	5.0.0
scikit-learn	1.6.1
numpy	2.1.3
pandas	2.2.3
rouge_score	installed (ROUGE-L is additionally implemented inline; see below)
matplotlib	figures only

Data

Dataset	CUAD v1, Hugging Face <code>theatticusproject/cuad</code>
Files used	<code>CUAD_v1/CUAD_v1.json</code> (contexts, character offsets); <code>CUAD_v1/master_clauses.csv</code> (exploratory analysis only)
Contracts	510; 41 categories; 20,910 (contract, category) rows
Note	The repository’s default <code>load_dataset()</code> configuration exposes only the raw PDFs and cannot be trained on (Section 4.3).

Seeds and splits

- Global seed 42 for the split, all three trainings, and every bootstrap.
- The split is *drawn once* from `numpy.random.default_rng(42).permutation`, saved to `artifacts/split.json`, and loaded from disk by every subsequent script, so no script re-derives it.
- Additional presence-model seeds 43 and 44 (Section 5.4.2); dataset-split seeds 0–19 for Section 5.4.4.
- Bootstrap: 10,000 resamples, `default_rng(42)`, resampling unit = contract.

Artifact fingerprints

The first 16 characters of the SHA-256 hash of each file (October 2026 models), so a reader can check that they have the same files:

<code>CUAD_v1.json</code>	<code>ed0b77d85bdf4014</code>
<code>artifacts/split.json</code>	<code>177e2ef48dfab9cd</code>
<code>artifacts/baseline.pkl</code>	<code>903e527500409af2</code>
<code>outputs/presence_mil/final/model.safetensors</code>	<code>a5482a0302ae2c7c</code>
<code>outputs/span/final/model.safetensors</code>	<code>380a233a3c03e052</code>
<code>outputs/summarizer/final/model.safetensors</code>	<code>9fcb6bdf777d033c</code>

Commands, and which table each regenerates

The four `stage` scripts are run from the `notebooks/` folder, exactly as listed in `retrain/README.md`; every other command works from the project folder. Rows marked “August” are the August 2026 control runs.

Command	Regenerates
<code>python3 ../retrain/smoke_test.py</code>	Checks that the four saved models load and run
<code>python3 ../retrain/stage2a_presence_train.py</code>	Split, TF-IDF baseline and presence transformer (≈ 50 min)
<code>python3 ../retrain/stage2a_presence_test.py</code>	Tables 5.1–5.4, Figures 5.1–5.3 (≈ 50 min)
<code>python3 ../retrain/stage2b_span_train.py, then _test.py</code>	Span model and its scores in Table 5.5 ($\approx 25 + 2$ min)
<code>python3 ../retrain/stage1_summarizer_train.py, then _test.py</code>	Summarizer and its template score in Table 5.6 ($\approx 13 + 3$ min)
<code>python3 retrain/analysis_oct2026/make_probs.py</code>	Saves the presence scores in the form the analysis scripts below read (run once after the presence test)
<code>python3 retrain/analysis_oct2026/span_ci.py</code>	Intervals in Table 5.5
<code>python3 retrain/analysis_oct2026/summ_pilot.py</code>	Tables 5.6 and 5.7
<code>python3 retrain/analysis_oct2026/e2e.py</code>	Tables 5.8 and 5.9
<code>python3 retrain/analysis_oct2026/risk_threshold_calib.py</code>	Table 5.10, Figure 5.4
<code>python3 retrain/analysis_oct2026/error_examples.py</code>	Table 5.11
<code>python3 retrain/analysis_oct2026/rare_tail.py</code>	Table 5.12
<code>python3 retrain/analysis_oct2026/pooling_ablation.py</code>	Table 5.13 (≈ 55 min)
<code>python3 retrain/analysis_oct2026/bootstrap_ci.py</code>	Table 5.14
<code>python3 analysis/presence_run.py --tag s43 --seed 43</code>	Seeds 43 and 44 in Table 5.15 (August; repeat for 44)
<code>python3 analysis/presence_run.py --tag full53k --max-train 0</code>	Table 5.16 (August; ≈ 140 min)
<code>python3 analysis/compare_runs.py</code>	Differences in Table 5.16 (August)
<code>python3 analysis/split_variance.py</code>	Table 5.17
<code>python3 analysis/lf_one.py longformer 4096 1 train</code>	Table 4.5 (one row per run)
<code>python3 analysis/app_bench.py</code>	Table 4.6
<code>python3 retrain/verify_report.py</code>	Checks the reported numbers in this report against the result files; prints 0 mismatched when they agree
<code>python3 review/agreement.py <sheets></code>	Reviewer agreement and Cohen's/Fleiss' κ once the risk-table review sheets are returned
<code>tectonic project_report/main.tex</code>	This report

Known determinism caveats

- MPS training is not bit-reproducible, even run to run on the same machine and library versions. The October 2026 re-training of every model with identical code and seed reproduced the August scores to within about 0.003 (presence micro-F1 0.6943 $\rightarrow$ 0.6924 for the transformer; span token-F1 0.763 $\rightarrow$ 0.764), not exactly.
- ROUGE-L in `summ_pilot.py` is computed by an inline longest-common-subsequence implementation (matching `test.ipynb`) rather than by the `rouge_score` package, so the two are not interchangeable.
- The retrieval-ceiling measurement of Section 4.2.1 was computed over all 510 contracts, test contracts included (Section 6.3).

Appendix C

Repository and Artifact Layout

```
clausify-p3/ (github.com/afnanmz168/clausify-p3) -- research deliverable
|- project_report/ -- this report (LaTeX, CSE400 template)
| |- main.tex / main.pdf
| |- chapters/ core/ appendix/ bibliography/ images/
|- paper/ -- IEEE-format paper (LaTeX + figures)
|- IEEE_Paper_Clausify.pdf
|- data/
| |- train_80.csv -- 80% split (16,728 rows, 408 contracts)
| |- test_20.csv -- 20% split (4,182 rows, 102 contracts)
|- notebooks/
| |- train.ipynb -- training notebook (EDA -> 3 models)
| |- test.ipynb -- STANDALONE evaluation notebook
| |- build_artifacts.py -- retrains & saves all artifacts
| |- artifacts/ -- split.json, baseline.pkl
| |- outputs/ -- presence, span, summarizer checkpoints
|- retrain/ -- October 2026 re-training & re-testing
| |- README.md -- exact terminal commands
| |- stage2a_* stage2b_* stage1_* -- train/test scripts, logs, results
| |- smoke_test.py, make_figures.py
| |- analysis_oct2026/ -- analyses re-run on the new models
| |- verify_report.py -- checks report numbers against results
|- analysis/
| |- bootstrap_ci.py -- cluster bootstrap, presence models
| |- span_ci.py -- cluster bootstrap, span extraction
| |- rare_tail.py -- rare-category AND-rule breakdown
| |- e2e.py -- end-to-end pipeline evaluation
| |- summ_pilot.py -- two-reference-set summarizer pilot
| |- presence_run.py -- presence training at any data budget
| |- longformer_bench.py -- sparse-attention cost benchmark
| |- diverse_refs.json -- 150 clause-specific pilot references
|- review/
| |- risk_taxonomy_review_sheet.csv -- face-validity review instrument
| |- agreement.py -- reviewer agreement (kappa)
|- demo_clause.txt -- 18-clause demonstration contract

clausify-p3-app/ (github.com/afnanmz168/clausify-p3-app)
|- app.py -- Streamlit UI
|- model_utils.py -- inference: all four models, clause mode
|- features.py -- file upload, highlighting, reports
|- calibrate_clause_mode.py -- clause-mode calibration (+ .json)
|- cat_questions.json -- 41 category->question strings
|- sample_contract.txt / test_contract.txt
|- tests/test_app.py -- 38 end-to-end checks
```